

KeepKey Firmware Test Report

Firmware 7.15.0 | 2026-09-22 23:53 | 327/330 PASSED, 3 skipped (withheld)

Scope: this report is a curated catalog of 330 tests. The CI run executed 1262 (456 of them native firmware unit tests). Absence from this report is NOT evidence that a feature is untested -- check the JUnit artifacts.

Sections

C Core - Device Lifecycle -- 31/31 passed
K Seed Generation Hardening (7.15) [NEW] -- 12/12 passed
B Bitcoin -- 33/33 passed
E Ethereum -- 23/26 passed
R Ripple (XRP) -- 3/3 passed
A Cosmos (ATOM) -- 3/3 passed
P Osmosis [NEW] -- 8/8 passed
H THORChain -- 4/4 passed
M Maya Protocol -- 6/6 passed
O EOS -- 4/4 passed
W Nano -- 1/1 passed
V EVM Clear-Signing [NEW] -- 86/86 passed
G Hive [NEW] -- 38/38 passed
S Solana [NEW] -- 26/26 passed
T TRON [NEW] -- 6/6 passed
N TON [NEW] -- 8/8 passed
Y Zcash Transparent -- 4/4 passed
Z Zcash Shielded (Orchard) [NEW] -- 25/25 passed
D BIP-85 Child Derivation [NEW] -- 6/6 passed

C. Core - Device Lifecycle

Fundamental device security operations. Every firmware version must pass these tests. A failure here is an absolute release blocker - these protect seed generation, backup, recovery, and access control.

User Flow

WIPE: Erases all keys and settings, returns to factory state
RESET: Generates cryptographic entropy -> BIP-39 mnemonic displayed on OLED only
RECOVERY: Cipher-based entry (scrambled keyboard on OLED) prevents keyloggers
PIN: Randomized grid on OLED, user enters position not digit
PASSPHRASE: Additional BIP-39 word, empty string = default wallet

Tests: 31/31 -- ALL PASSED

✓C1 test_wipe_device

Wipe device (test_msg_wipedevice.py)
Erases all keys, PIN, settings. Device shows "WIPE DEVICE - Do you want to erase your private keys and settings?" on OLED. User must press button to confirm. After wipe, device is uninitialized - no operations work until a new seed is loaded or generated.

IMPORT RECOVERY SENTENCE



Importing is not recommended unless you understand the risks. Do you want to import recovery sentence?

DEBUG_LINK

WIPE DEVICE



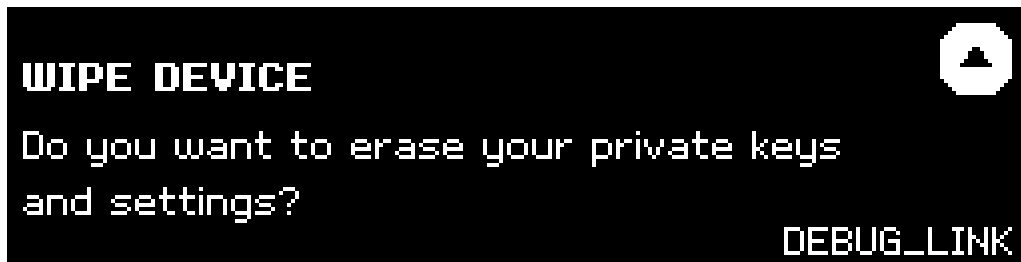
Do you want to erase your private keys and settings?

DEBUG_LINK

✓C2 test_reset_device

Generate new seed (test_msg_resetdevice.py)

Device generates 256 bits of entropy from hardware RNG, converts to BIP-39 mnemonic, and displays words on OLED one page at a time. Words are NEVER sent to the host. User writes them down as their backup.



✓C3 test_reset_device_pin

Generate seed with PIN (test_msg_resetdevice.py)

Same as C2 but also sets a PIN. PIN is entered twice for confirmation via the randomized 3x3 grid on OLED. Verifies PIN is stored and required for subsequent operations.

OLED needed: PIN entry grid

✓C4 test_failed_pin

PIN mismatch rejects setup (test_msg_resetdevice.py)

If the user enters different PINs during confirmation, the device rejects the setup. This prevents accidentally setting a PIN the user cannot reproduce.

OLED needed: PIN mismatch warning

✓C5 test_already_initialized

Reject reset on initialized device (test_msg_resetdevice.py)

An already-initialized device must refuse reset without a wipe first. Prevents accidental seed replacement which would strand funds on the old seed.

✓C6 test_load_device_1

Load 12-word mnemonic (debug) (test_msg_loaddevice.py)

Debug-only operation: loads a known 12-word mnemonic for testing. In production, seeds can only be generated on-device or recovered via cipher entry.

✓C7 test_load_device_2

Load 18-word mnemonic (debug) (test_msg_loaddevice.py)

Tests 18-word BIP-39 mnemonic support (192 bits of entropy).

✓C8 test_load_device_3

Load 24-word mnemonic (debug) (test_msg_loaddevice.py)

Tests 24-word BIP-39 mnemonic support (256 bits of entropy, maximum security).

✓C9 test_load_device_utf

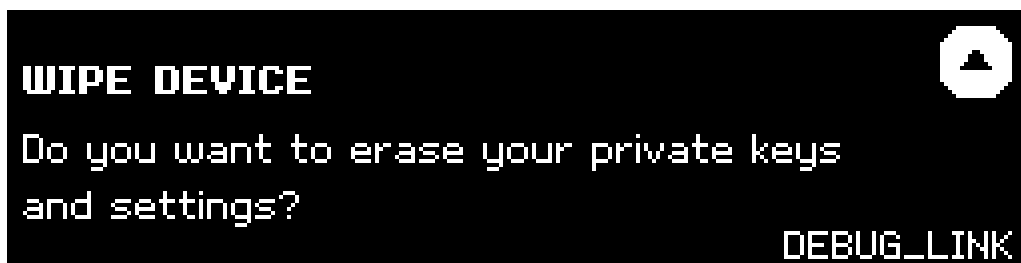
Load with UTF-8 device label (test_msg_loaddevice.py)

Verifies the device handles non-ASCII characters in labels without corruption.

✓C10 test_nopin_nopassphrase

Cipher recovery (no PIN) (test_msg_recoverydevice_cipher.py)

Recovery via scrambled keyboard on OLED. The letter grid is randomized per-character, so even a compromised host cannot determine which letters the user selected. After all words are entered, device verifies BIP-39 checksum and reconstructs the seed.



✓C11 test_pin_passphrase

Cipher recovery with PIN + passphrase (test_msg_recoverydevice_cipher.py)

Same recovery flow as C10 but also sets PIN and enables passphrase protection during the recovery process.

OLED needed: Cipher + PIN entry

✓C12 test_character_fail

Invalid character rejection (test_msg_recoverydevice_cipher.py)

Verifies the cipher entry rejects characters that cannot form any BIP-39 word prefix.

✓C13 test_backspace

Backspace during cipher entry (test_msg_recoverydevice_cipher.py)

User can correct mistakes during word entry without restarting recovery.

✓C14 test_reset_and_recover

Full reset then recover cycle (test_msg_recoverydevice_cipher.py)

End-to-end test: generate seed -> write down words -> wipe -> recover from words -> verify same addresses are derived. Proves the backup/restore cycle works.

✓C15 test_wrong_number_of_words

Wrong word count rejected (test_msg_recoverydevice_cipher.py)

BIP-39 only allows 12, 18, or 24 words. Other counts are rejected immediately.

✓C16 test_correct_same

Dry-run recovery matches (test_msg_recoverydevice_cipher_dryrun.py)

User can verify their backup without wiping the device. Dry-run recovers the seed in memory and compares to the active seed. If they match, user knows their backup is valid.

✓C17 test_correct_notsame

Dry-run detects wrong backup (test_msg_recoverydevice_cipher_dryrun.py)

If the entered words produce a different seed, the device warns the user. This catches transcription errors in the backup before an emergency.

✓C18 test_incorrect

Dry-run rejects bad entry (test_msg_recoverydevice_cipher_dryrun.py)

Invalid words or checksum failure during dry-run are reported to the user.

✓C19 test_set_pin

Set new PIN (test_msg_changepin.py)

Transitions from no-PIN to PIN-protected. The randomized 3x3 grid prevents screen recording attacks - the attacker sees button presses but not which digit they map to.

OLED needed: PIN entry grid

✓C20 test_change_pin

Change existing PIN (test_msg_changepin.py)

Requires entering the current PIN first (proving knowledge), then setting a new one.

✓C21 test_remove_pin

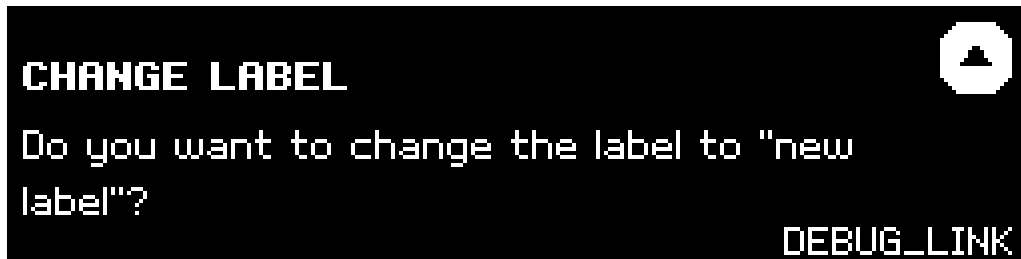
Remove PIN protection (test_msg_changepin.py)

User can disable PIN if physical security is sufficient. Requires current PIN to remove.

✓C22 test_apply_settings

Change label and language (test_msg_applysettings.py)

Device label appears on OLED during confirmation screens. Helps identify devices when a user has multiple KeepKeys.



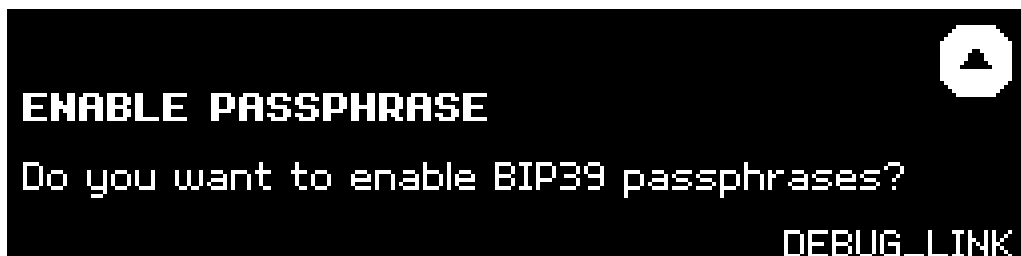
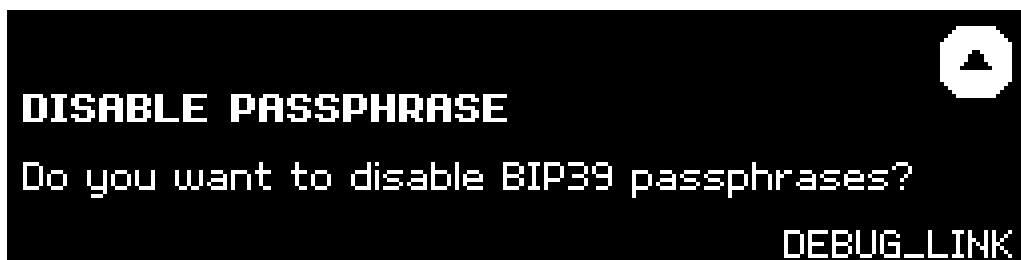
(2 OLED frames captured, showing 1)

✓C23 test_apply_settings_passphrase

Toggle passphrase protection (test_msg_applysettings.py)

Enables/disables BIP-39 passphrase. When enabled, every operation prompts for a passphrase.

Different passphrases derive completely different wallets from the same seed.





ENABLE PASSPHRASE

Do you want to enable BIP39 passphrases?

DEBUG_LINK

(4 OLED frames captured, showing 3)

✓C24 test_clearsession

Clear session state (test_msg_clearsession.py)

Clears cached PIN, passphrase, and session data. Next operation requires re-authentication.

✓C25 test_ping

Ping with button confirmation (test_msg_ping.py)

Basic connectivity test. Verifies the device processes messages and button confirmation works.

✓C26 test_ping_format_specifier_sanitize

Sanitize format specifiers (test_msg_ping.py)

Security test: printf-style format specifiers in ping message must not cause crashes or information leaks. Verifies input sanitization.

✓C27 test_entropy

Hardware RNG audit budget and lock policy (test_msg_getentropy.py)

Proves a fresh initialized, PIN-protected, locked device still requires confirmation; then proves an uninitialized device returns exactly 8 x 8192 bytes (64 KiB) without a press, with exact lengths, unique blocks, and conservative catastrophic-failure health checks. The next request must restore confirmation. These checks detect a stuck or grossly biased source; they are not a statistical certification of the hardware RNG.

✓C28 test_encrypt

Symmetric key encryption (test_msg_cipherkeyvalue.py)

Derives a symmetric key from the HD tree and encrypts data. Used for password manager integrations and encrypted communication.

✓C29 test_decrypt

Symmetric key decryption (test_msg_cipherkeyvalue.py)

Reverse of C28. Verifies encrypt/decrypt round-trips correctly.

✓C30 test_sign

Sign identity challenge (SSH/GPG) (test_msg_signidentity.py)

Signs an identity challenge for SSH login or GPG key derivation. Derives a key from the identity URI and signs the challenge.

✓C31 test_invalid_bip39_word_rejected

BIP-39 invalid word rejected during cipher recovery (test_msg_recoverydevice_cipher.py)

Enter a non-BIP-39 word ("zz") during cipher recovery with enforce_wordlist=True. Firmware must reject immediately with Failure instead of silently accepting.



Word not in wordlist

DEBUG_LINK



Word not in wordlist

DEBUG_LINK

K. Seed Generation Hardening (7.15) [NEW]

The 7.15 changes to how a seed comes into existence: user-supplied dice entropy folded in on-device, and the PIN key-derivation rewrap. These ran green from the first 7.15 RC but appeared nowhere in this report, because the catalog could not reference native firmware unit tests at all and nobody had catalogued the two new pyk cases. Absent evidence read as absent coverage during an RC audit, which is exactly the failure this section exists to prevent.

User Flow

DICE: user rolls a d6 on-device; short press advances 1-6, long press commits, undo backs out. The roll string is hashed and the digest confirmed on the OLED before it is mixed in.

MIX: `int_entropy = SHA256(int_entropy || rolls)`, folded in BEFORE the host `EntropyRequest`, so the device commits to its own contribution first and the host cannot choose the seed.

ABORT: any aborted reset must disarm `EntropyAck`, or a later host `EntropyAck` would derive a seed from `sha256(0*32 || host_bytes)` -- entirely host-chosen. That is K2.

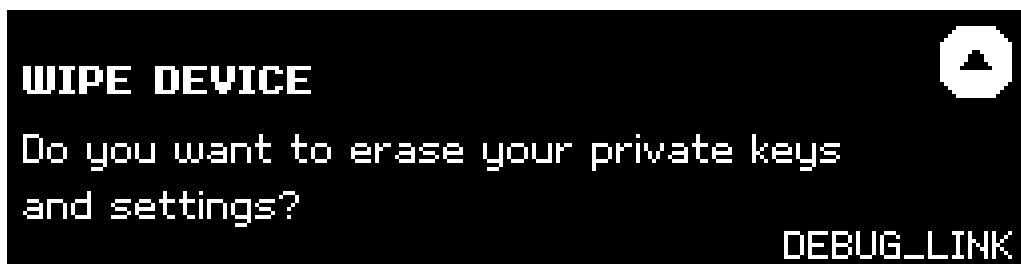
PIN KDF: a v16 storage blob must still unlock and then rewrap to v19, or the upgrade bricks.

Tests: 12/12 -- ALL PASSED

✓K1 test_reset_device_dice

Dice entropy end-to-end (`test_msg_resetdevice.py`)

Drives the full on-device dice flow over `DebugLink`: 99 rolls injected in chunks with undo exercised, extras past the cap dropped. Asserts the device-computed digest equals SHA256 of exactly the expected roll string, then derives the mnemonic from the post-mix internal entropy and compares -- which is what proves the rolls actually reached the seed rather than being collected and discarded.



✓K2 test_reset_reentry_disarms_entropy_ack

Aborted reset disarms `EntropyAck` (`test_msg_resetdevice.py`)

Regression for a host-chosen-seed hole: `reset_init` aborts left awaiting `_entropy` set from an earlier run while zeroing `int_entropy`, so a following `EntropyAck` derived the seed from host bytes alone. Arms a reset, re-enters with dice, cancels, and asserts the next `EntropyAck` is refused with "Not in Reset mode" and the device stays uninitialized.

✓K3 RollsForStrength

Roll count per seed strength (`Dice.py`)

d6 carries $\log_2(6)=2.585$ bits, so 128/192/256-bit seeds need 50/75/99 rolls (the Coldcard convention). A short count would silently weaken the seed.

✓K4 MixZeroEntropyVector

Mix known-answer vector (zero entropy) (`Dice.py`)

`SHA256(0x00*32 || "123456")` against a hardcoded digest. Pins the mix construction so a refactor cannot quietly change how dice enter the seed.

✓K5 MixNonZeroEntropyVector

Mix known-answer vector (non-zero entropy) (`Dice.py`)

Same construction with a non-zero starting entropy buffer, pinned to a hardcoded digest.

✓K6 MixDependsOnRolls

Different rolls produce different entropy (`Dice.py`)

Two mixes differing only in the final roll must diverge. Catches a mix that ignores its roll argument -- the failure mode where dice appear to work and contribute nothing.

✓K7 MixUsesExactCount

Only the counted rolls contribute (`Dice.py`)

Bytes past the declared roll count must not affect the result, so uninitialized tail bytes of the roll buffer can never leak into seed material.

✓K8 PinKdfRewrapsToActiveVersionAfterCorrectPin

Correct PIN unlocks and rewraps to the ACTIVE KDF (`Storage.py`)

The migration path for the hardened PIN KDF: an existing device must still unlock with its current PIN, and any rewrap must target whatever KDF the build actually has enabled. Renamed from `PinKdfV16RewrapsToV19AfterCorrectPin` because it is no longer v19-specific -- the test now asserts BOTH sides of the `STORAGE_PIN_KDF_V19` gate, so it is meaningful in the shipping build where v19 is off. If this regressed, every upgrading device would be locked out of its own seed.

✓K8b PinUnlocksAfterRebootUnderV17

The PIN still opens the wallet after a reboot (Storage.py)

The whole round trip in device order: create, set a PIN, serialize the V17 record as storage_commit() does, reload into fresh state as a boot would, unlock, decrypt. Every other storage test stays in RAM, and the wallet lockout this guards against lived exactly on the serialize/reboot boundary -- a wrap the persisted record could not describe, so the next boot derived the wrong KDF and every PIN failed.

✓K9 PinKdfV2FlagsVersionedInV19

KDF version flag is recorded in v19 (Storage.py)

The new KDF is marked in the storage version band, so firmware can tell which derivation a blob was written with instead of guessing.

✓K10 StorageUpgrade_Normal

Normal storage upgrade path (Storage.py)

Baseline upgrade across storage versions with policies and cache preserved.

✓K11 NoopSecMigrate

Idempotent security migration (Storage.py)

Re-running the migration on already-migrated storage must be a no-op rather than a second rewrap.

B. Bitcoin

Bitcoin is the primary chain and most extensively tested. Covers legacy P2PKH, P2SH-wrapped SegWit, native SegWit (bech32), and Taproot (P2TR). Transaction signing validates that the device correctly displays every output address and amount, calculates fees, detects change outputs, and resists output substitution attacks. Also covers UTXO forks sharing BTC signing code.

User Flow

ADDRESS: Derive key from BIP-32 path -> display on OLED with QR code -> user verifies against host

SIGN TX: Device shows each output (full address + amount) -> shows fee -> user confirms -> signs

MESSAGE: Show text on OLED -> user confirms -> signs with address-specific key (EIP-191 equivalent)

Tests: 33/33 -- ALL PASSED

✓B1 test_btc

Derive BTC legacy address (test_msg_getaddress.py)

Derives a P2PKH (1...) address from standard BIP-44 path m/44'/0'/0'/0/0. Verifies the address matches the expected value from the test mnemonic.

✓B2 test_ltc

Derive Litecoin address (test_msg_getaddress.py)

LTC uses the same derivation as BTC with coin_type=2. Verifies L... address format.

✓B3 test_tbtc

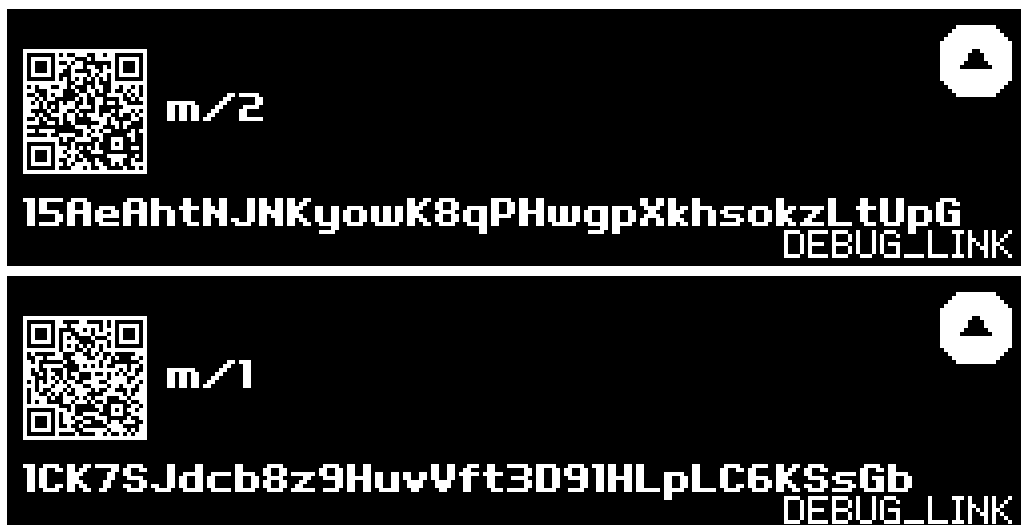
Derive testnet address (test_msg_getaddress.py)

Testnet addresses use different version bytes (m/n prefix). Important for development testing.

✓B4 test_show

Show BTC address on OLED (test_msg_getaddress_show.py)

Address displayed on OLED with QR code for visual verification. User compares the address shown on the trusted device display against the host application. This is the primary defense against address substitution attacks by compromised hosts.





m/3



1CmzyJp9w3NafXMSEFH4SLYUPAVCSUrrJ5
DEBUG_LINK

✓B5 test_show_multisig_3

Show 3-of-3 multisig address (test_msg_getaddress_show.py)

Multisig addresses require all co-signer xpubs. Device displays the P2SH multisig address derived from all provided public keys.



Multisig (2 of 3)



3E7GDtuHqnqPmDgwH59pVC7AvySiSkbibz
DEBUG_LINK



Multisig (2 of 3)



3E7GDtuHqnqPmDgwH59pVC7AvySiSkbibz
DEBUG_LINK



Multisig (2 of 3)



3E7GDtuHqnqPmDgwH59pVC7AvySiSkbibz
DEBUG_LINK

✓B6 test_show_segwit

Show SegWit P2SH address (test_msg_getaddress_segwit.py)

P2SH-wrapped SegWit (3... prefix). Backwards compatible with legacy wallets while getting SegWit fee savings.



⚡ Testnet Account #0
Change Address #0



2N1LGaGg836mqSQqiuUBLfcyGBhyZbremDX
DEBUG_LINK

✓B7 test_show_segwit

Show native SegWit bech32 (test_msg_getaddress_segwit_native.py)

Native SegWit (bc1q... prefix). Lowest fees, modern address format. Verifies bech32 encoding.



Testnet Account #0
Address #0



tb1qqzv60m9ajw8drqulta4ld4gfx0rdh82un5s65s
DEBUG_LINK

WARNING



Wrong address path for selected coin.
Continue at your own risk!

DEBUG_LINK

✓B8 test_btc

Get BTC xpub (test_msg_getpublickey.py)
Exports the extended public key for a derivation path. Used by wallet software to derive addresses and monitor balances without the device connected.

✓B9 test_one_one_fee

Sign basic BTC transaction (test_msg_signtx.py)
Simplest case: one input, one output. Device displays "Send X BTC to [address]" with the full recipient address (no truncation), then shows the fee. Verifies the signed transaction is valid.

Send 0.0038 BTC to
1MJ2tj2ThBE62zXbBYA5ZaN3fdve5CPAz1



DEBUG_LINK

TRANSACTION



Do you want to send 0.0039 BTC from your
wallet? This includes a transaction fee of
0.0001 BTC.

DEBUG_LINK

✓B10 test_one_two_fee

Sign BTC tx with change (test_msg_signtx.py)
One input, two outputs (payment + change). Device must identify the change output (same xpub tree) and only display the payment output to the user.

Send 0.003 BTC to
1MJ2tj2ThBE62zXbBYA5ZaN3fdve5CPAz1



DEBUG_LINK



Send 0.0008 BTC to
1CK7Sjdcb8z9HuvVft3D91HLpLC6KSsGb

DEBUG_LINK

(3 OLED frames captured, showing 2)

✓B11 test_two_two

Sign multi-input BTC tx (test_msg_signtx.py)

Two inputs, two outputs. Verifies correct fee calculation across multiple inputs.

✓B12 test_spend_coinbase

Sign coinbase spend (test_msg_signtx.py)

Spending a coinbase (mining reward) output. Coinbase outputs have special maturity rules.

✓B13 test_lots_of_outputs

Sign tx with many outputs (test_msg_signtx.py)

Stress test with many recipients. Each output is displayed individually on the OLED.

✓B14 test_fee_too_high

Reject excessive fee (test_msg_signtx.py)

If the fee exceeds a safety threshold, the device shows a prominent warning. Protects against fat-finger errors or malicious fee manipulation.



CONFIRM FEE

Really spend 0.0025 BTC on fees? Except in
times of high network congestion, fees
should be less than 100 sat/byte.

DEBUG_LINK



Send 0.0014 BTC to
1MJ2tj2ThBE62zXbBYA5ZaN3fdve5CPAz1

DEBUG_LINK



TRANSACTION

Do you want to send 0.0039 BTC from your
wallet? This includes a transaction fee of
0.0025 BTC.

DEBUG_LINK

✓B15 test_not_enough_funds

Reject insufficient funds (test_msg_signtx.py)

If inputs don't cover outputs + fee, the device refuses to sign.

✓B16 test_p2sh

Sign P2SH transaction (test_msg_signtx.py)

Pay-to-Script-Hash output. Used for multisig and complex scripts.

✓B17 test_attack_change_outputs

Detect output substitution (test_msg_signtx.py)

Security test: the host attempts to substitute the change output address between the first and second signing pass. Device must detect the mismatch and refuse.

✓B18 test_send_p2sh

Sign SegWit P2SH tx (test_msg_signtx_segwit.py)

SegWit transaction with P2SH-wrapped inputs. Different signing algorithm (BIP-143).

✓B19 test_send_mixed

Sign mixed legacy+SegWit tx (test_msg_signtx_segwit.py)
Transaction with both legacy and SegWit inputs in the same transaction.

✓B20 test_send_p2tr_only

Create a Taproot P2TR output (test_msg_signtx_p2tr.py)
Pays from SegWit inputs to a P2TR output. This exercises P2TR output parsing and display, but does not exercise a Schnorr key-path spend.



Send 0.00251476 BTC to
bc1plsk660nud549q0p5hnlc0ldvgvxxaamce
k68r8zsgp9xmhjypp4s2d4xdc

DEBUG_LINK



TRANSACTION

Do you want to send 0.00253856 BTC from
your wallet? This includes a transaction fee
of 0.0000238 BTC.

DEBUG_LINK

✓B21 test_send_p2tr

Sign a Taproot key-path spend (test_msg_signtx_taproot.py)
Spends a BIP-86 P2TR input using BIP-341 SIGHASH_DEFAULT and a BIP-340 Schnorr signature. The 64-byte witness is compared byte-for-byte with an independently computed reference value. The complete 153-byte transaction is then parsed as BIP-144 and must consume every byte, proving the witness stack and the 4-byte locktime footer actually reached the host rather than only the signature field.



TRANSACTION

Do you want to send 0.001 BTC from your
wallet? This includes a transaction fee of
0.0001 BTC.

DEBUG_LINK



Send 0.0009 BTC to
1BitcoinEaterAddressDontSendf59kuE

DEBUG_LINK

✓B22 test_send_p2tr_with_change

Sign P2TR with device-derived change (test_msg_signtx_taproot.py)
Derives m/86'/0'/0'/1/0 on-device, emits a P2TR change output, and verifies the Schnorr witness against an independent BIP-340/341 reference. The complete 196-byte transaction is parsed as BIP-144 and must consume every byte, and the change output is matched as a full value/length/script triple.



TRANSACTION

Do you want to send 0.0006 BTC from your
wallet? This includes a transaction fee of
0.0001 BTC.

DEBUG_LINK



Send 0.0005 BTC to
1BitcoinEaterAddressDontSendMessage

DEBUG_LINK

✓B23 test_send_mixed_p2tr_and_legacy

Sign mixed Taproot and legacy inputs (test_msg_signtx_taproot.py)
Commits the P2TR signature to both inputs, including the legacy prevout amount and scriptPubKey, while independently verifying the resulting Schnorr witness. The complete 301-byte transaction is parsed as BIP-144; the Taproot input must carry a single 64-byte stack item and the legacy input its empty 0x00 witness.

✓B24 test_mixed_p2tr_requires_every_input_amount

Reject incomplete mixed Taproot commitments (test_msg_signtx_taproot.py)
Fails closed when any input amount is absent, preventing the device from producing a valid Schnorr signature over an incomplete BIP-341 commitment.

✓B25 test_mixed_p2tr_rejects_wrong_legacy_amount

Reject a tampered legacy prevout amount (test_msg_signtx_taproot.py)
Fetches the actual legacy prevout and rejects a host-provided amount that differs by one satoshi, preventing a false BIP-341 commitment in a mixed-input transaction.

✓B26 test_show_taproot_address

Show BIP-86 address on OLED (test_msg_getaddress_taproot.py)
Displays the complete bech32m Taproot receive address and QR code on the trusted device screen for host-independent verification.



m/86'/0'/0'/0/0

bc1p5cyxnuxmeuwvkwfem96lqzszd02n6xdcjrs20
cac6ygjjwudpxqkedrcr

DEBUG_LINK



✓B27 test_sign

Sign message with BTC key (test_msg_signmessage.py)
Signs arbitrary text with a BTC address key. Used for proof-of-ownership and login.



SIGN MESSAGE

This is an example of a signed message.

DEBUG_LINK

✓B28 test_sign

Sign message with SegWit key (test_msg_signmessage_segwit.py)
Message signing with P2SH-SegWit address key.



SIGN MESSAGE

This is an example of a signed message.

DEBUG_LINK

✓B29 test_sign

Sign message with bech32 key (test_msg_signmessage_segwit_native.py)
Message signing with native SegWit address key.



SIGN MESSAGE

This is an example of a signed message.

DEBUG_LINK

✓B30 test_message_verify

Verify signed message (test_msg_verifymessage.py)
Device verifies a message signature against a BTC address.

✓B31 test_send_bitcoin_gold_nochange

Sign Bitcoin Gold tx (test_msg_signtx_bgold.py)
BTG fork uses same signing code with different chain parameters.

✓B32 test_send_dash

Sign Dash transaction (test_msg_signtx_dash.py)
Dash special transaction types (InstantSend-compatible).

✓B33 test_one_one_fee

Sign Groestlcoin tx (test_msg_signtx_grs.py)
GRS uses Groestl hash instead of SHA-256d for tx hashing.



TRANSACTION

Do you want to send 0.00210016 GRS from
your wallet? This includes a transaction fee
of 0.00000192 GRS.

DEBUG_LINK



Send 0.00209824 GRS to
FtM4zAn9aVYgHgxmamWBgWPYzsb6RhvkA9

DEBUG_LINK

E. Ethereum

Ethereum covers native ETH transfers, ERC-20 tokens, EIP-1559 gas, personal message signing (EIP-191), and contract interactions. The device displays checksummed addresses (EIP-55) and gas parameters. Amount UNIT rule: values below 1 gwei (1e9 wei) show as raw "Wei" (there is no smaller human unit to scale to); values at or above 1 gwei show 18-decimal-scaled ETH (or the chain-native ticker on other EVM chains). Some tests below use small conformance-vector amounts (e.g. 10 wei) for deterministic-signature pinning - their OLED frames legitimately show raw "Wei", not a display bug.

User Flow

ETH TRANSFER: Show "Send X ETH to 0x..." -> show gas -> confirm -> sign with secp256k1
ERC-20: Decode transfer(to,amount) from contract data -> show token name + amount
EIP-1559: Show maxFeePerGas + maxPriorityFeePerGas (not legacy gasPrice)
MESSAGE: EIP-191 prefix -> show text on OLED -> sign with ETH key

Tests: 26

✓E1 test_ethereum_getaddress

Derive ETH address (test_msg_ethereum_getaddress.py)
Standard m/44'/60'/0'/0/0 derivation. EIP-55 checksum address.
OLED needed: ETH address

✓E2 test_ethereum_signtx_nodata

Sign ETH transfer (test_msg_ethereum_signtx.py)
Simple value transfer with no contract data. Device shows recipient + amount + gas.



TRANSACTION

Send 12.34567890123456789 from your
wallet, paying up to 400000000 Wei for gas?

[DEBUG_LINK](#)



SEND

Send 12.34567890123456789 to
0x1D1C328764A41BdA0492b66bAA30C4A339FF8

SEF

[DEBUG_LINK](#)

(5 OLED frames captured, showing 2)

✓E3 test_ethereum_signtx_data

Sign ETH tx with contract data (test_msg_ethereum_signtx.py)

Transaction with data field (contract call). Device shows data as hex since it cannot decode arbitrary ABI without metadata.



CONFIRM ETHEREUM DATA

6162636465666768696A6B6C6D6E6F70

6162636465666768... 256 bytes

[DEBUG_LINK](#)



CONFIRM ETHEREUM DATA

6162636465666768696A6B6C6D6E6F70

6162636465666768... 256 bytes

[DEBUG_LINK](#)



SEND

Send 12.34567890123456789 to
0x1D1C328764A41BdA0492b66bAA30C4A339FF8

SEF

[DEBUG_LINK](#)

(11 OLED frames captured, showing 3)

✓E4 test_ethereum_signtx_nodata_eip155

Sign ETH with EIP-155 replay protection (test_msg_ethereum_signtx.py)

Chain ID embedded in signature v value to prevent cross-chain replay attacks.



TRANSACTION

Send 0.1 from your wallet, paying up to
0.00042 for gas?

[DEBUG_LINK](#)

SEND



Send 0.1 to

0x8eA7a3fccC211ED48b763b4164884DDbcF3b0

A98

[DEBUG_LINK](#)

SEND



Send 0.1 to

0x8eA7a3fccC211ED48b763b4164884DDbcF3b0

A98

[DEBUG_LINK](#)

(5 OLED frames captured, showing 3)

✓E5 test_ethereum_eip_1559

Sign EIP-1559 transaction (test_msg_ethereum_signtx.py)

Type 2 transaction with base fee + priority fee. Device shows both gas parameters.

TRANSACTION



Send 0.006 ETH from your wallet, paying up
to 0.002247419875336155 ETH for gas?

[DEBUG_LINK](#)

SEND



Send 0.006 ETH to

0xfc0Cc6E85dFf3D75e3985e0CB83B090cfD49

8dd1

[DEBUG_LINK](#)

✓E5b test_eip1559_chunked_data_signature_recovers_to_device_address

Sign EIP-1559 with data > 1024 B (chunked transmission) (test_msg_ethereum_signtx_chunked_data_eip1559.py)

Regression for an access-list ordering bug in firmware/ethereum.c - when data exceeded the 1024-byte single-chunk threshold, the empty access-list byte (0xC0) was hashed between data chunks instead of after them, producing a non-canonical pre-image. The signature recovered to a wrong-but-deterministic address and the broadcast tx was dropped from the mempool. Fixed in 7.14.1.

✓E6 test_ethereum_signtx_knownerc20_eip_1559

Sign known ERC-20 (EIP-1559) (test_msg_ethereum_signtx.py)

Known token (in firmware token list) via EIP-1559. Shows human-readable token name + amount.

SEND



Send Unknown token value to

0x574BbB36871bA6b78E27f4B4dCFb76eA0091

880B

[DEBUG_LINK](#)



TRANSACTION

Send the tokens from your wallet, paying
up to 400 Wei for gas?

[DEBUG_LINK](#)

✓E7 test_ethereum_sign_message

Sign personal message (test_msg_ethereum_message.py)

EIP-191 personal_sign. Device shows the message text on OLED for user to verify before signing.



SIGN ETHEREUM MESSAGE HEX 1/1

eef81f6d2517f420fc0f59684fb3d4cb9ebdf0
bb3a8f6075b9c5e1f3210231f0

[DEBUG_LINK](#)

✓E8 test_ethereum_sign_bytes

Sign raw bytes (test_msg_ethereum_message.py)

Signs arbitrary bytes (displayed as hex on OLED).

✓E9 test_ethereum_verify_message

Verify ETH signed message (test_msg_ethereum_message.py)

Device-side verification of EIP-191 signed messages.

✓E10 test_approve_some

ERC-20 approve specific amount (test_msg_signtx_ethereum_erc20.py)

Token approval for a specific amount. Device shows spender address + approved amount.



APPROVE

Approve withdrawal of up to 42 CVC by
0x1D1C328764A41BdA0492b66bAA30C4A339FF8
5EF?

[DEBUG_LINK](#)



TRANSACTION

Send the message from your wallet, paying
up to 400 Wei for gas?

[DEBUG_LINK](#)

✓E11 test_approve_all

ERC-20 approve unlimited (test_msg_signtx_ethereum_erc20.py)

MAX_UINT256 approval. Device shows "UNLIMITED" warning since this grants infinite spending.



APPROVE

Unlock full CVC balance for withdrawal by
0x1D1C328764A41BdA0492b66bAA30C4A339FF8
5EF?

[DEBUG_LINK](#)



TRANSACTION

Send the message from your wallet, paying
up to 400 Wei for gas?

DEBUG_LINK

✓E12 test_generate

MakerDAO generate DAI (test_msg_ethereum_makerdao.py)
Complex DeFi contract interaction (MakerDAO CDP).

✓E13 test_sign_salarywithdrawal

Sablier salary withdrawal (test_msg_ethereum_sablier.py)
Streaming payment protocol contract call.

✓E14 test_sign_0x_swap_ETH_to_ERC20

0x swap ETH to ERC-20 (test_msg_ethereum_erc20_0x_signtx.py)
DEX aggregator swap via 0x protocol.

✓E15 test_sign_execTx

Contract function call (test_msg_ethereum_cfunc.py)
Generic contract call signing.

✓E16 test_ethereum_sign_typed_data_hash

EIP-712 typed-data hash signing (legacy, no on-device display) (test_sign_typed_data.py)
The legacy endpoint receives two host-computed 32-byte hashes, so firmware keeps it behind
AdvancedMode and cannot show readable WHO/WHAT. Structured formats such as x402 EIP-3009 use
the separate device-parsed path proven by E16b.

✓E16b test_ethereum_sign_x402_eip3009

x402 EVM EIP-3009 payment clear-signs structured data (test_sign_typed_data.py)
The device computes the EIP-712 hashes itself and displays the Base Sepolia USDC domain plus
every TransferWithAuthorization field: payer, recipient, exact value, validity window and
nonce. AdvancedMode stays OFF; the facilitator pays gas but cannot alter the signed destination
or amount.



EIP-712 VALUE

0xf3746613c2d920b5fdabc0856f2aeb2d4f88
ee6037b8cc5d04a71a4462f13480

DEBUG_LINK



EIP-712 FIELD

name

DEBUG_LINK



EIP-712 VALUE

USDC

DEBUG_LINK

(23 OLED frames captured, showing 3)

-- E17 test_sign_uni_approve_liquidity_ETH

Uniswap V2 add-liquidity approve (pending) (test_msg_ethereum_erc20_uniswap_liquidity.py)
PENDING, disclosed: known emulator limitation - an approve to an unknown (non-registry) token
contract cannot complete against the kkemu emulator (matches the sibling add/remove-liquidity
skips below); the device-firmware path is not in question, only CI emulator coverage.
Real-device testing is unaffected.

-- E18 test_sign_uni_add_liquidity_ETH

Uniswap V2 add liquidity ETH+token (pending) (test_msg_ethereum_erc20_uniswap_liquidity.py)

PENDING, disclosed: same emulator limitation as E17 - a daily-driver LP-deposit flow with no PDF proof on this build; tracked for real-device verification.

-- E19 test_sign_uni_remove_liquidity_ETH

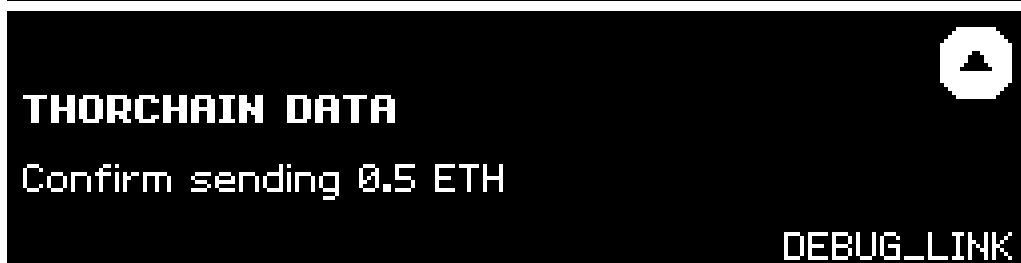
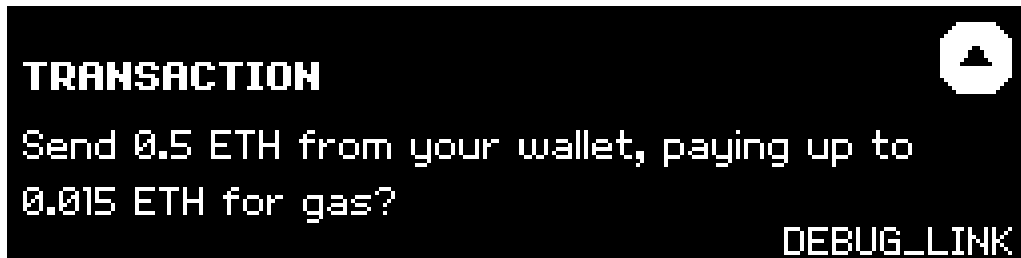
Uniswap V2 remove liquidity ETH+token (pending) (test_msg_ethereum_erc20_uniswap_liquidity.py)

PENDING, disclosed: same emulator limitation as E17.

✓E20 test_deposit_legacy_selector

THORChain router deposit() (legacy selector) (test_msg_ethereum_thorchain_deposit.py)

Cross-chain swap via the THORChain router contract - a daily-driver EVM<->THORChain swap path, natively decoded (asset/amount/memo) without clear-sign metadata. The native amount shown is the signed msg.value (the ABI amount word is a router-ignored hint and is never displayed as the send amount).

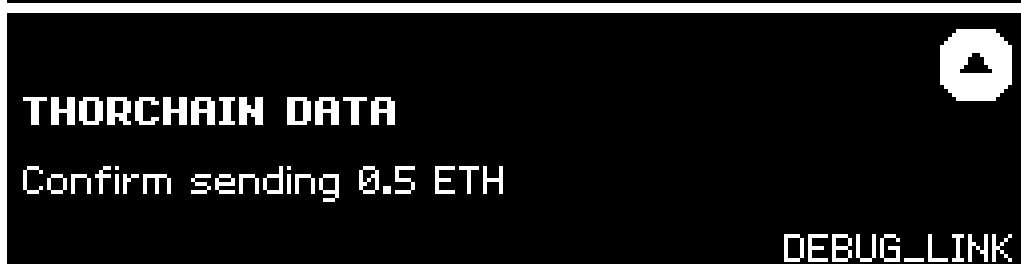
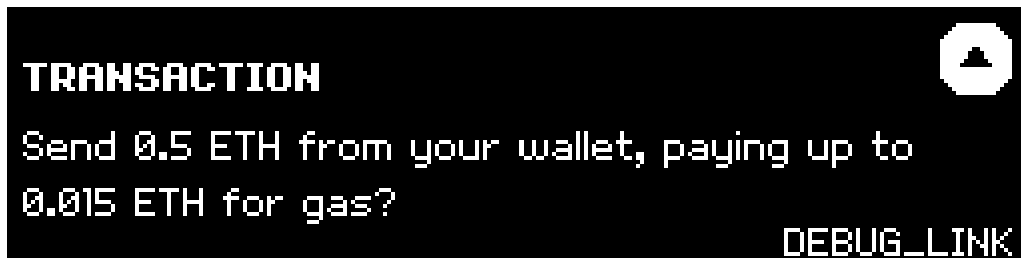


(9 OLED frames captured, showing 2)

✓E21 test_deposit_with_expiry_selector

THORChain router depositWithExpiry() (test_msg_ethereum_thorchain_deposit.py)

Newer router selector variant with an expiry field; same native decode path. The ABI memo length word is read from the calldata (not assumed 64 bytes) and the padded memo must end exactly at the calldata end.

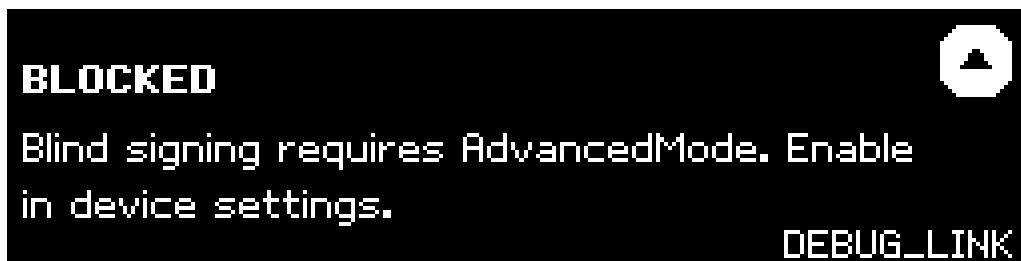
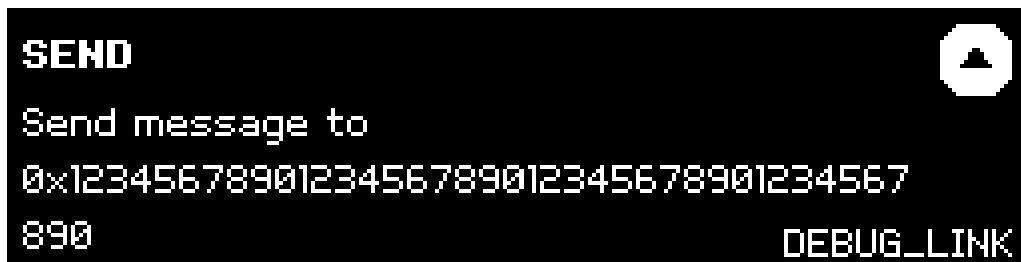


(9 OLED frames captured, showing 2)

✓E22 test_deposit_with_expiry_non_thor_address_blind_sign_blocked

THORChain router call to a non-pinned address is blind-sign gated (test_msg_ethereum_thorchain_deposit.py)

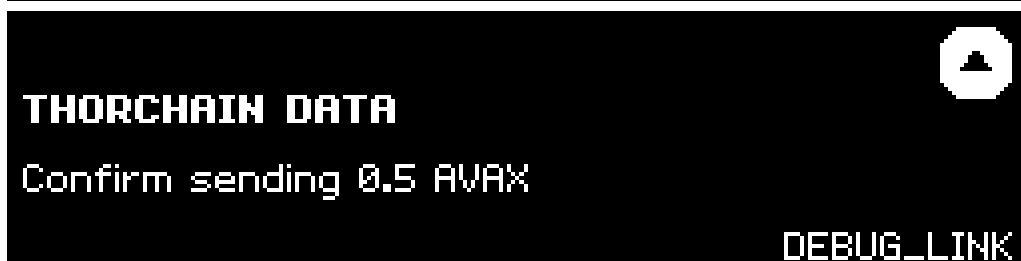
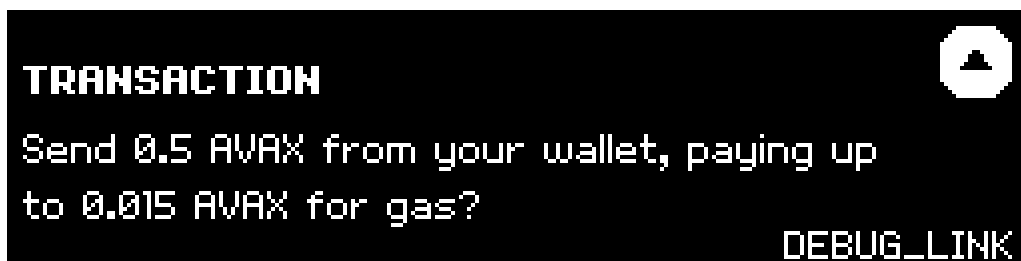
WHY it can be trusted: the router CONTRACT ADDRESS is pinned; a call shaped like a THORChain deposit but sent to an unpinned address is refused native decoding and falls through to the ordinary blind-sign gate instead of being silently native-decoded - the fix for the router-spoofing / blind-sign-bypass class of attack.



(3 OLED frames captured, showing 2)

✓E23 test_deposit_with_expiry_avalanche_router

THORChain deposit on Avalanche clear-signs (per-chain router pin) (test_msg_ethereum_thorchain_deposit.py)
THORChain deploys its router at a DIFFERENT address on every EVM chain, so the pin is (chain_id, address) together. Before the chain scope, only mainnet deposits ever matched and an AVAX->ETH swap fell into the blind-sign gate. The Avalanche C-Chain router (00dc61..f1d4) is verified live against THORChain /inbound_addresses; the native amount screen shows msg.value with the CHAIN's ticker (AVAX), and the signature is ECDSA-recovered against the host-built pre-image over chainId 43114.



(9 OLED frames captured, showing 2)

✓E24 test_deposit_unpinned_chain_blind_sign_blocked

Deposit on an unpinned chain is blind-sign gated (test_msg_ethereum_thorchain_deposit.py)
The mainnet router ADDRESS on a chain with no pinned router (BSC) must not inherit the deposit UX - the same address on another chain may hold unrelated attacker code. Falls to the AdvancedMode gate; rejection is pre-UI (no frame).

R. Ripple (XRP)

XRP Ledger support for the third-largest cryptocurrency by market cap. XRP uses a unique account-based model (not UTXO) with 20 XRP minimum reserve. Amounts are denominated in drops (1 XRP = 1,000,000 drops). Destination tags are required for exchange deposits to route funds to the correct account. The device displays the full rAddress (34 chars starting with r) and converts drop amounts to human-readable XRP values.

User Flow

ADDRESS: Derive from m/44'/144'/0'/0/0 -> display full rAddress + QR on OLED

SIGN: Host sends Payment tx (destination, amount, fee, destination_tag) -> device shows XRP amount + recipient

FEE: XRP requires a minimum fee (currently 10 drops). Device validates fee is within bounds.

Tests: 3/3 -- ALL PASSED

✓R1 test_ripple_get_address

Derive XRP address (test_msg_ripple_get_address.py)

Standard m/44'/144'/0'/0/0 derivation.

OLED needed: XRP address

✓R2 test_sign

Sign XRP payment (test_msg_ripple_sign_tx.py)

Payment with amount in drops (1 XRP = 1,000,000 drops).

SEND 

Send 100.000000 XRP to
rNaqKtKrMSupwZSzRckPf7S96DkimjkF4H,
with destination tag 123456? [DEBUG_LINK](#)

SEND 

Send 100 XRP to
rBKz5MC2iXdoS3XgnNSYmF69KTYo4NS3Ws? [DEBUG_LINK](#)

TRANSACTION 

Really send 100 XRP, with a transaction fee
of 0.1 XRP? [DEBUG_LINK](#)

(6 OLED frames captured, showing 3)

✓R3 test_ripple_sign_invalid_fee

Reject invalid fee (test_msg_ripple_sign_tx.py)

Fee outside acceptable range is rejected.

A. Cosmos (ATOM)

Cosmos Hub is the anchor chain for the Cosmos IBC ecosystem. Transactions use amino encoding (legacy Cosmos SDK format). The device supports MsgSend (transfers), MsgDelegate (staking to validators), and MsgWithdrawDelegatorReward (claiming staking rewards). Addresses use bech32 encoding with the cosmos1 prefix. Memo field is critical for exchange deposits and IBC transfers - the device displays it in full on the OLED for user verification.

User Flow

ADDRESS: Derive from m/44'/118'/0'/0/0 -> display cosmos1... bech32 address

SEND: Show recipient address + ATOM amount + memo on OLED -> user confirms

MEMO: Displayed in full - required for exchange deposits (e.g. numeric account ID)

Tests: 3/3 -- ALL PASSED

✓A1 test_standard

Derive Cosmos address (test_msg_cosmos_getaddress.py)

Bech32 cosmos1... address from m/44'/118'/0'/0/0.

✓A2 test_cosmos_sign_tx

Sign Cosmos send (test_msg_cosmos_signtx.py)

MsgSend with amount + recipient display.

Send 0.1 ATOM to
cosmos18vhdczjut44gpsy804crfhnd5nq00
3nz0nf20v 

[DEBUG_LINK](#)

COSMOS ACCOUNT #0



Sign this Cosmos transaction on
cosmoshub-2? It includes a fee of 5000
uATOM and 200000 gas.

DEBUG_LINK

✓A3 test_cosmos_sign_tx_memo

Sign Cosmos with memo (test_msg_cosmos_signtx.py)
Memo field displayed for exchange deposit tags.

Send 8.675309 ATOM to
cosmos18vhdczjut44gpsy804crfhnd5nq00
3nz0nf20v



DEBUG_LINK

MEMO

test memo



DEBUG_LINK

COSMOS ACCOUNT #0



Sign this Cosmos transaction on
cosmoshub-2? It includes a fee of 5000
uATOM and 200000 gas.

DEBUG_LINK

P. Osmosis [NEW]

Osmosis is the Cosmos-ecosystem DEX, signed with the same amino encoding as Cosmos Hub and derived from the same coin type (118). 7.15.0 CHANGED how every Osmosis amount is drawn: the confirm screens formatted with atof() + "%.6f", and a float carries only ~7 significant decimal digits, so a large transfer was displayed ROUNDED on the screen the user approves (123456789.123456 OSMO rendered as 123456792.000000). The signature was always over the correct amount - the error was confined to the display, which is the half a hardware wallet exists to get right. Amounts now use bounded decimal-string formatting; native uosmo is canonical uint64, unknown denominations remain exact base-unit strings, and every long signed asset is renderer-paged before signing.

User Flow

SEND: recipient + OSMO amount rendered from integer base units, never a float
PRECISION: 15-significant-digit amounts display exactly, not rounded to 7
UNKNOWN DENOM: shown as raw base units - the device does not guess a decimal point

Tests: 8/8 -- ALL PASSED

✓P1 test_osmosis_sign_tx

Sign Osmosis send (test_msg_osmosis_signtx.py)
Baseline MsgSend: recipient and a whole-OSMO amount on the confirm screen.



SEND AMOUNT

1.500000 OSMO

[DEBUG_LINK](#)



SEND TO

osmo1Scenya0tr7nm3tz2wn3h3zwkht2rxrq7
g9ypmq

[DEBUG_LINK](#)



OSMOSIS ACCOUNT #0

Sign this Osmosis transaction on osmosis-1?
It includes a fee of 800 uOSMO and 290000
gas.

[DEBUG_LINK](#)

✓P2 test_osmosis_send_amount_beyond_float_precision

Amount beyond float precision displays exactly (test_msg_osmosis_signtx.py)

The regression this section exists for: 123456789123456 uosmo needs 15 significant digits. The old float path drew 123456792.000000 OSMO over a transaction moving 123456789.123456 OSMO. The captured frame is the evidence.



SEND AMOUNT

123456789.123456 OSMO

[DEBUG_LINK](#)



SEND TO

osmo1Scenya0tr7nm3tz2wn3h3zwkht2rxrq7
g9ypmq

[DEBUG_LINK](#)



OSMOSIS ACCOUNT #0

Sign this Osmosis transaction on osmosis-1?
It includes a fee of 800 uOSMO and 290000
gas.

[DEBUG_LINK](#)

✓P3 test_osmosis_send_subunit_amount

Sub-unit amount keeps its tail (test_msg_osmosis_signtx.py)

500 uosmo is 0.000500 OSMO - no integer part and six decimals; it must not collapse to 0 or lose the trailing digits.



SEND AMOUNT

0.000500 OSMO

DEBUG_LINK



SEND TO

osmo15cenyao0tr7nm3tz2wn3h3zkwkht2rxrq7
g9ypmq

DEBUG_LINK



OSMOSIS ACCOUNT #0

Sign this Osmosis transaction on osmosis-1?
It includes a fee of 800 uOSMO and 290000
gas.

DEBUG_LINK

✓P4 test_osmosis_send_denom_is_committed_to_the_signature

Direct-wire denomination is committed (test_msg_osmosis_signtx.py)
Two otherwise-identical raw MsgSend requests using uosmo and uatom produce different
signatures, proving the reviewed denomination is part of the signed payload rather than a
hardcoded display-only label.

✓P5 test_osmosis_send_rejects_noncanonical_wire_amounts

Noncanonical and overflowing uosmo are refused (test_msg_osmosis_signtx.py)
Raw-wire 01, -1, leading-space and UINT64 overflow values are rejected before any
display/signature divergence can occur.

✓P6 test_osmosis_swap_max_fields_are_fully_paged

Maximum Swap fields are fully paged (test_msg_osmosis_signtx.py)
Two maximum-size 68-character IBC denominations plus 32-digit amounts force the exact OLED
renderer across separate bounded screens. The full ordered input and minimum-output sequence is
captured before the signature is returned.



SWAP INPUT

12345678901234567890123456789012

ibc/AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA DEBUG_LINK



MINIMUM OUTPUT

12345678901234567890123456789012

ibc/AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA DEBUG_LINK



CONFIRM POOL ID

1

DEBUG_LINK



OSMOSIS ACCOUNT #0

Sign this Osmosis transaction on osmosis-1?
It includes a fee of 800 uOSMO and 290000
gas.

DEBUG_LINK

(4 OLED review screens, in order)

✓P7 test_osmosis_amount_is_committed_to_the_signature

Displayed amount is in the digest (test_msg_osmosis_signtx.py)
Two sends differing only in amount produce different signatures, so the confirm screen is bound to what is signed rather than decorative.

✓P8 test_osmosis_signing_is_deterministic

Deterministic nonces (RFC6979) (test_msg_osmosis_signtx.py)
Identical input yields an identical signature; a mismatch is a key-recovery risk, not a cosmetic one.

H. THORChain

THORChain is a decentralized cross-chain liquidity protocol. Native RUNE transactions use amino encoding with thor1... bech32 addresses. The memo field is the critical security element - it encodes the entire swap/LP instruction (e.g. "SWAP:BTC.BTC:bc1q..." or "=:ETH.ETH:0x..."). A compromised host could substitute the memo destination address to steal funds. The device displays the full memo text on OLED so users can verify the swap destination, pool, and parameters before signing. THORChain also supports LP add/remove operations and deposits.

User Flow

ADDRESS: Derive from m/44'/931'/0'/0/0 -> display thor1... bech32 address
SEND: Show RUNE amount + recipient + full memo text on OLED
SWAP MEMO: "SWAP:BTC.BTC:bc1q..." - user verifies destination chain, asset, and receiving address
LP MEMO: "ADD:BTC.BTC:thor1..." or "WITHDRAW:BTC.BTC:10000" - user verifies pool and basis points

Tests: 4/4 -- ALL PASSED

✓H1 test_thorchain_get_address

Derive THORChain address (test_msg_thorchain_getaddress.py)
Bech32 thor1... address.

✓H2 test_thorchain_sign_tx

Sign THORChain tx - raw memo paged in full (7 memo variants) (test_msg_thorchain_signtx.py)
Native RUNE transfer. The COMPLETE raw memo is paged on the OLED (MEMO 1/N..N/N, 72-char pages) as the sole memo gate - no structured summary can hide trailing content, and a reject on any page aborts signing. The frames below show every page for each routed memo shape (SWAP/s/=/ADD/a/+ and bare-pool).



Send 0.0001 to
tthor1jvt443rvhq5h8yrna55yjysvhtju0el7l
dnwwy

DEBUG_LINK



ASSET

rune

[DEBUG_LINK](#)



MEMO

SWAP:ETH.USDT-0xdac17f958d2ee523a22062
06994597c13d831ec7:0x41e5560054824ea6b0
732e656e3ad64e20e94e45:420

[DEBUG_LINK](#)



THORCHAIN ACCOUNT #0

Sign this THORChain transaction on
thorchain? Additional network fees apply.

[DEBUG_LINK](#)



Send 0.0001 to
tthor1jvt443rvhq5h8yrna55yjysvhtju0el7l
dnwwy

[DEBUG_LINK](#)



ASSET

rune

[DEBUG_LINK](#)



MEMO

s:ETH.USDT-0xdac17f958d2ee523a220620699
4597c13d831ec7:0x41e5560054824ea6b0732e
656e3ad64e20e94e45:

[DEBUG_LINK](#)



THORCHAIN ACCOUNT #0

Sign this THORChain transaction on
thorchain? Additional network fees apply.

[DEBUG_LINK](#)



Send 0.0001 to
tthor1jvt443rvhq5h8yrna55yjysvhtju0el7l
dnwwy

[DEBUG_LINK](#)



ASSET

rune

[DEBUG_LINK](#)



MEMO

=:ETH.USDT-0xdac17f958d2ee523a220620699
4597c13d831ec7::420

[DEBUG_LINK](#)



THORCHAIN ACCOUNT #0

Sign this THORChain transaction on
thorchain? Additional network fees apply.

[DEBUG_LINK](#)



Send 0.0001 to
tthor1jvt443rvhq5h8yrna55yjysvhtju0el7l
dnwwy

[DEBUG_LINK](#)



ASSET

rune

[DEBUG_LINK](#)



MEMO

SWAP:BTC.BTC

[DEBUG_LINK](#)



THORCHAIN ACCOUNT #0

Sign this THORChain transaction on
thorchain? Additional network fees apply.

[DEBUG_LINK](#)



Send 0.0001 to
tthorljvt443rvhq5h8yrna55yjysvhtju0el7l
dnwwy

[DEBUG_LINK](#)



ASSET

rune

[DEBUG_LINK](#)



MEMO

ADD:ETH.USDT-0xdac17f958d2ee523a2206206
994597c13d831ec7:0x41e5560054824ea6b073
2e656e3ad64e20e94e45

[DEBUG_LINK](#)



THORCHAIN ACCOUNT #0

Sign this THORChain transaction on
thorchain? Additional network fees apply.

[DEBUG_LINK](#)



Send 0.0001 to
tthorljvt443rvhq5h8yrna55yjysvhtju0el7l
dnwwy

[DEBUG_LINK](#)



ASSET

rune

[DEBUG_LINK](#)

MEMO



a:ETH.USDT-0xdac17f958d2ee523a220620699
4597c13d831ec7:0x41e5560054824ea6b0732e
656e3ad64e20e94e45

DEBUG_LINK

THORCHAIN ACCOUNT #0



Sign this THORChain transaction on
thorchain? Additional network fees apply.

DEBUG_LINK

Send 0.0001 to
tthorljvt443rvhq5h8yrna55yjysvhtju0el7l
dnwwy



DEBUG_LINK

ASSET



rune

DEBUG_LINK

MEMO



+:ETH.USDT-0xdac17f958d2ee523a220620699
4597c13d831ec7:0x41e5560054824ea6b0732e
656e3ad64e20e94e45

DEBUG_LINK

THORCHAIN ACCOUNT #0



Sign this THORChain transaction on
thorchain? Additional network fees apply.

DEBUG_LINK

(28 OLED review screens, in order)

✓H3 test_sign_btc_eth_swap

Sign BTC->ETH swap (test_msg_thorchain_signtx.py)

Cross-chain swap via THORChain memo routing.

CONFIRM FEE



Really spend 0.0039 BTC on fees? Except in times of high network congestion, fees should be less than 100 sat/byte. [DEBUG_LINK](#)

THORCHAIN SWAP



Confirm to
0x41e5560054824ea6b0732e656e3ad64e20e94e45 [DEBUG_LINK](#)

THORCHAIN SWAP



Confirm limit 420 [DEBUG_LINK](#)

(5 OLED frames captured; showing 3)

✓H4 test_thorchain_sign_tx_deposit

Sign THORChain deposit (test_msg_2thorchain_signtx.py)

LP deposit transaction (MsgDeposit): asset, amount and the full memo are displayed from the exact bytes being signed.

Send 0.50994 THOR.RUNE to
thor1ls33ayg26kmltw7jy55p32ghjna09zp74t4az



[DEBUG_LINK](#)

MEMO



SWAP:BNB.BNB:bnb12splwpg8jenr9pjw3dwc5rr35t8792y8pc4mtf:348953501 [DEBUG_LINK](#)

THORCHAIN ACCOUNT #0



Sign this THORChain transaction on
thorchain? Additional network fees apply. [DEBUG_LINK](#)

M. Maya Protocol

Maya Protocol is a THORChain fork providing cross-chain liquidity with its native CACAO token. Uses identical amino transaction format and memo-based routing as THORChain but with maya1... bech32 addresses. Maya bridges assets between Bitcoin, Ethereum, THORChain, Dash, and Kujira. The same memo security considerations apply - the device must display the full memo for swap destination verification.

User Flow

ADDRESS: Derive from m/44'/931'/0'/0/0 -> display maya1... bech32 address

SEND: Show CACAO amount + recipient + full memo on OLED

SWAP: Same memo format as THORChain with Maya-specific pool routing

Tests: 6/6 -- ALL PASSED

✓M1 test_mayachain_get_address

Derive Maya address (test_msg_mayachain_getaddress.py)

Bech32 maya1... address.

✓M2 test_sign_btc_eth_swap

Sign BTC-ETH swap via Maya (test_msg_mayachain_signtx.py)

Cross-chain swap via Maya memo routing (BTC OP_RETURN side).

✓M3 test_sign_eth_add_liquidity

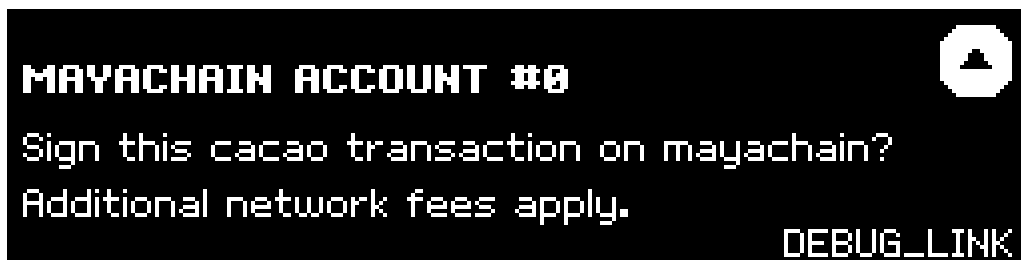
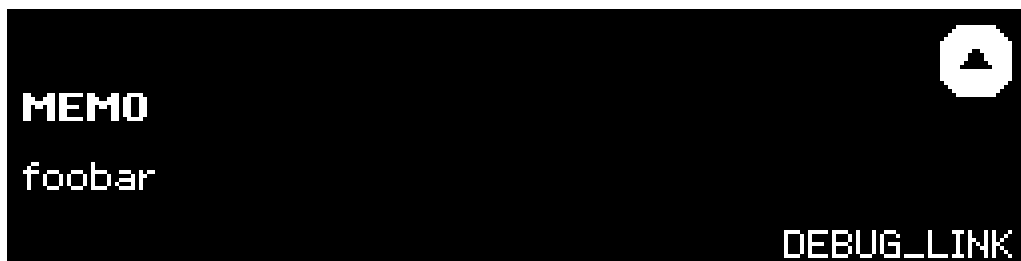
Add liquidity via Maya router (EVM side) (test_msg_mayachain_signtx.py)

depositWithExpiry() to the firmware-pinned Maya router; the signature is recovered to the device signer over the exact calldata.

✓M4 test_mayachain_sign_tx

Sign native CACAO MsgSend - raw memo paged (test_msg_mayachain_signtx.py)

Native CACAO transfer. Signature verified host-side against the amino sign-doc digest (account/chain/fee/memo/amount/addresses all bound) and the known device pubkey - no frozen vectors to go stale. The complete raw memo is paged on the OLED (thorchain_confirm_full_memo is the sole memo gate for native MAYA too).

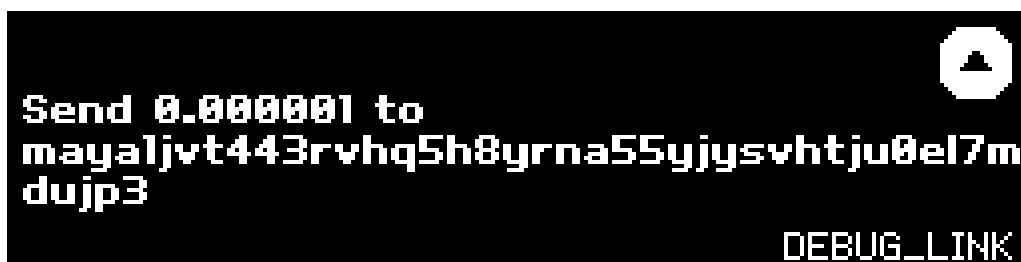


(4 OLED frames captured; showing 2)

✓M5 test_mayachain_sign_tx_memos

Native memo variants - every routed shape paged in full (test_msg_mayachain_signtx.py)

Each memo shape MAYA routes on (SWAP/s/= /ADD/a/+ and bare-pool) signs, each signature is bound to its exact memo bytes via the sign-doc digest, and every page of every memo is displayed (frames below, in order).





ASSET

cacao

[DEBUG_LINK](#)



MEMO

SWAP:ETH.USDT-0xdac17f958d2ee523a22062
06994597c13d831ec7:0x41e5560054824ea6b0
732e656e3ad64e20e94e45:420

[DEBUG_LINK](#)



MAYACHAIN ACCOUNT #0

Sign this cacao transaction on mayachain?
Additional network fees apply.

[DEBUG_LINK](#)



Send 0.000001 to
magaljvt443rvhq5h8yrna55yjysvhtju0el7m
dujp3

[DEBUG_LINK](#)



ASSET

cacao

[DEBUG_LINK](#)



MEMO

s:ETH.USDT-0xdac17f958d2ee523a220620699
4597c13d831ec7:0x41e5560054824ea6b0732e
656e3ad64e20e94e45:

[DEBUG_LINK](#)



MAYACHAIN ACCOUNT #0

Sign this cacao transaction on mayachain?
Additional network fees apply.

[DEBUG_LINK](#)



Send 0.000001 to
magaljvt443rvhq5h8yrna55yjysvhtju0el7m
dujp3

[DEBUG_LINK](#)



ASSET

cacao

[DEBUG_LINK](#)



MEMO

=:ETH.USDT-0xdac17f958d2ee523a220620699
4597c13d831ec7::420

[DEBUG_LINK](#)



MAYACHAIN ACCOUNT #0

Sign this cacao transaction on mayachain?
Additional network fees apply.

[DEBUG_LINK](#)



Send 0.000001 to
magaljvt443rvhq5h8yrna55yjysvhtju0el7m
dujp3

[DEBUG_LINK](#)



ASSET

cacao

[DEBUG_LINK](#)



MEMO

SWAP:BTC.BTC

[DEBUG_LINK](#)



MAYACHAIN ACCOUNT #0

Sign this cacao transaction on mayachain?
Additional network fees apply.

[DEBUG_LINK](#)



Send 0.000001 to
magaljvt443rvhq5h8yrna55yjysvhtju0el7m
dujp3

[DEBUG_LINK](#)



ASSET

cacao

[DEBUG_LINK](#)



MEMO

ADD:ETH.USDT-0xdac17f958d2ee523a2206206
994597c13d831ec7:0x41e5560054824ea6b073
2e656e3ad64e20e94e45

[DEBUG_LINK](#)



MAYACHAIN ACCOUNT #0

Sign this cacao transaction on mayachain?
Additional network fees apply.

[DEBUG_LINK](#)



Send 0.000001 to
magaljvt443rvhq5h8yrna55yjysvhtju0el7m
dujp3

[DEBUG_LINK](#)



ASSET

cacao

[DEBUG_LINK](#)

MEMO



a:ETH.USDT-0xdac17f958d2ee523a220620699
4597c13d831ec7:0x41e5560054824ea6b0732e
656e3ad64e20e94e45

[DEBUG_LINK](#)

MAYACHAIN ACCOUNT #0



Sign this cacao transaction on mayachain?
Additional network fees apply.

[DEBUG_LINK](#)

Send 0.000001 to
mayaljvt443rvhq5h8yrna55yjysvhtju0el7m
dujp3



[DEBUG_LINK](#)

ASSET



cacao

[DEBUG_LINK](#)

MEMO



+:ETH.USDT-0xdac17f958d2ee523a220620699
4597c13d831ec7:0x41e5560054824ea6b0732e
656e3ad64e20e94e45

[DEBUG_LINK](#)

MAYACHAIN ACCOUNT #0



Sign this cacao transaction on mayachain?
Additional network fees apply.

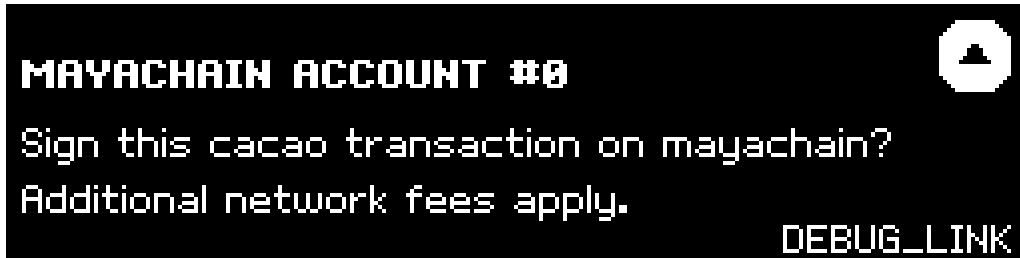
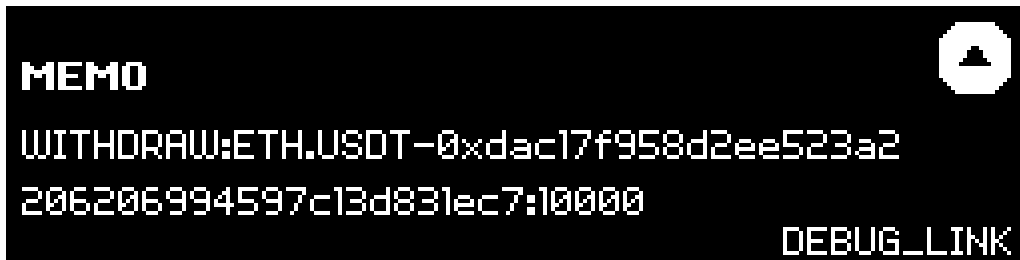
[DEBUG_LINK](#)

(28 OLED review screens, in order)

✓M6 test_mayachain_remove_liquidity

Native WITHDRAW memo (test_msg_mayachain_signtx.py)

WITHDRAW:pool:basis-points memo paged in full; signature digest-verified.



(4 OLED frames captured, showing 2)

O. EOS

EOS chain support with action-based transaction model. Unlike UTXO or account-based chains, EOS transactions contain a list of actions, each targeting a specific smart contract. The device displays each action individually for user review. Covers the core eosio system actions: token transfers, CPU/NET bandwidth delegation, block producer voting, and account authority management (updateauth, linkauth, newaccount). EOS uses a unique account name system (12-char names) instead of addresses.

User Flow

PUBKEY: Derive EOS public key from m/44'/194'/0'/0/0 (EOS format with EOS prefix)
SIGN TX: Host sends action list -> device displays each action with contract + data -> signs
STAKING: delegatebw/undelegatebw for CPU/NET resource management
GOVERNANCE: voteproducer to select block producers

Tests: 4/4 -- ALL PASSED

✓01 test_trezor

Derive EOS public key (test_msg_eos_getpublickey.py)
EOS public key from m/44'/194'/0'/0/0.

✓02 test_transfer

Sign EOS transfer (test_msg_eos_signtx.py)
eosio.token::transfer action.

✓03 test_delegatebw

Delegate bandwidth (test_msg_eos_signtx.py)
CPU/NET resource staking.

✓04 test_voteproducer

Vote for producer (test_msg_eos_signtx.py)
Block producer voting.

W. Nano

Nano uses a unique block-lattice architecture where each account has its own blockchain. Transactions are feeless and near-instant. The device validates balance encoding for Nano state blocks, which represent the entire account state (balance, representative, link) in a single block. Balance values use 128-bit raw amounts (1 Nano = 10³⁰ raw).

User Flow

ENCODE: Validate 128-bit balance representation for state block construction
STATE BLOCK: account + previous + representative + balance + link -> hash -> sign

Tests: 1/1 -- ALL PASSED

✓W1 test_encode_balance

Encode Nano balance (test_msg_nano_signtx.py)
Validates the 128-bit balance encoding used in Nano state blocks. Incorrect encoding would cause fund loss or invalid transactions on the block-lattice.

V. EVM Clear-Signing [NEW]

The purpose of clear-signing: instead of blind-signing an opaque hash, the device screen answers WHO / WHAT / WHY before the user approves. WHO = the validated contract address (full, never truncated) + attested protocol name. WHAT = the decoded method and its typed arguments in human terms (recipient address, "amount: 10.5 DAI" - not raw wei). WHY it can be trusted = a signer whose key the device trusts attested that this exact description matches this exact transaction, and the signature is REFUSED unless the signed digest equals the metadata's committed tx hash (fail-closed, replay-proof). NEW (phase 1): there is NO built-in "KeepKey says this is safe" key - every signer is loaded at runtime (LoadClearsignSigner, user-confirmed, RAM-only) and EVERY tx it describes is preceded by a warning naming the signer alias + fingerprint ("NOT verified by KeepKey"). The built-in warning-free path returns once the signer infra is hardened. The V9 flow below shows the full ordered review of a REAL Aave V3 supply() tx: the actual calldata (selector 0x617ba037 + asset + amount + onBehalfOf + referralCode, 132 bytes) is signed, and the metadata decodes it to protocol=Aave V3, asset=DAI, amount=10.5 DAI.

User Flow

LOAD SIGNER: LoadClearsignSigner -> on-device confirm (alias + fingerprint) -> RAM slot
WHO: warning (signer alias) + Contract: 0x... (full address) + protocol name
WHAT: Call: <method> + each decoded arg (ADDRESS / TOKEN_AMOUNT "10.5 DAI" / STRING)
WHY: signature refused unless signed digest == metadata tx_hash (replay-proof)
BLIND SIGN: No metadata + AdvancedMode off -> unknown contract data hard-rejected

Tests: 86/86 -- ALL PASSED

✓V1 test_valid_metadata_returns_verified

Valid metadata accepted (test_msg_ethereum_clear_signing.py)
Correctly signed metadata blob from a loaded signer is accepted. Device shows the clearsign warning (signer alias + fingerprint) then the decoded method + contract.
OLED needed: Clearsign warning (signer alias)

✓V2 test_wrong_key_returns_malformed

Wrong signing key rejected (test_msg_ethereum_clear_signing.py)
Metadata signed with wrong key is rejected as malformed.

✓V3 test_tampered_method_returns_malformed

Tampered method rejected (test_msg_ethereum_clear_signing.py)
Modified method name in blob fails signature check.

✓V4 test_tampered_contract_returns_malformed

Tampered contract rejected (test_msg_ethereum_clear_signing.py)
Modified contract address fails signature check.

✓V5 test_no_metadata_then_sign_unchanged

No metadata = blind sign path (test_msg_ethereum_clear_signing.py)
Without metadata, transaction goes through existing blind-sign path.

TRANSACTION

Send 1 ETH from your wallet, paying up to 0.00042 ETH for gas?

DEBUG_LINK

SEND

Send 1 ETH to 0xd8dA6BF26964aF9D7eEd9e03E53415037aA96045

DEBUG_LINK

✓V6 test_signature_verification

Signature verification math (test_msg_ethereum_clear_signing.py)
Unit test for the metadata blob signature algorithm.

✓V7 test_tampered_blob_fails_verification

Tampered blob fails (test_msg_ethereum_clear_signing.py)
Any byte change in the blob invalidates the signature.

✓V7a test_empty_payload_returns_malformed

Empty metadata payload rejected (test_msg_ethereum_clear_signing.py)
A zero-length blob classifies MALFORMED, never VERIFIED.

✓V7b test_truncated_payload_returns_malformed

Truncated metadata payload rejected (test_msg_ethereum_clear_signing.py)
A blob cut short of the minimum structural size classifies MALFORMED.

✓V7c test_extra_trailing_bytes_returns_malformed

Trailing garbage bytes rejected (test_msg_ethereum_clear_signing.py)
A blob with extra bytes appended past its declared structure classifies MALFORMED - the parser cannot be tricked by appended data.

✓V7d test_wrong_version_returns_malformed

Unknown version byte rejected (test_msg_ethereum_clear_signing.py)
A blob with a version byte the firmware does not recognize classifies MALFORMED rather than being guessed-parsed.

✓V7e test_zero_signature_returns_malformed

All-zero signature rejected (test_msg_ethereum_clear_signing.py)
A blob with a zeroed signature field classifies MALFORMED - an attacker cannot skip signing by leaving the field blank.

✓V7f test_empty_key_slot_returns_malformed

Metadata against an empty key slot rejected (test_msg_ethereum_clear_signing.py)
A blob referencing a signer slot with no key loaded classifies MALFORMED.

✓V8 test_ethereum_blind_sign_allowed

Blind sign permitted (AdvancedMode ON) (test_msg_ethereum_signtx.py)
Contract data with AdvancedMode enabled. Device allows signing. Blind-sign policy gating covered in 7.15.0+.

✓V9 test_binding_happy_path_signs_and_recovers

Full who/what/why review of a real Aave V3 supply() (test_msg_ethereum_clear_signing.py)
TX: to=0x7d27..c7a9 (Aave V3 Pool), data=0x617ba037 + asset(DAI) + amount(10.5e18) + onBehalfOf(0xd8dA..6045) + referralCode(0), chainId 1. METADATA decodes it to protocol="Aave V3", asset=0x6B17..1d0F, amount=10.5 DAI, onBehalfOf=0xd8dA..6045, bound to the exact sighash.
The OLED screens below are the full ordered review the user sees: warning -> Call: supply -> Contract -> protocol -> asset -> amount (10.5 DAI, decimal-scaled, NOT wei) -> onBehalfOf -> tx confirm. The signature then recovers to the device signer over THIS tx digest, proving the metadata was bound to this tx.

IDENTITY CI Test (0C2CB8B9) describes this tx. DEBUG_LINK	
CLEARSIGN Call: supply DEBUG_LINK	
SUPPLY Contract: 0x7d2768dE32b0b80b7a3454c06BdAc94A69Dc7A9 DEBUG_LINK	



SUPPLY

protocol:
Rave V3

[DEBUG_LINK](#)



SUPPLY

asset:
0x6B175474E89094C44D398b954EedaAC49527
1d0F

[DEBUG_LINK](#)



SUPPLY

amount:
10.5 DAI

[DEBUG_LINK](#)



SUPPLY

onBehalfOf:
0xd8dA6BF26964aF9D7eEd9e03E53415D37aA9
6045

[DEBUG_LINK](#)



SEND

Send message to
0x7d2768dE32b0b80b7a3454c06BdAc94A69D
Dc7A9

[DEBUG_LINK](#)



CONFIRM ETHEREUM DATA

617BA037000000000000000000000000000000
6B175474E89094C4... 132 bytes

[DEBUG_LINK](#)



TRANSACTION

Send the message from your wallet, paying
up to 0.004 ETH for gas?

[DEBUG_LINK](#)

✓V10 test_replay_rejected_when_digest_differs

Replay reject (binding enforced) (test_msg_ethereum_clear_signing.py)

Metadata committed to tx A; signing tx B (same contract/selector/chain, different calldata) is refused at send_signature with "Metadata does not match signed transaction".

SUPPLY



onBehalfOf:

0xd8dA6BF26964aF9D7eEd9e03E53415D37aA9

6045

DEBUG_LINK

SUPPLY



Contract:

0x7d2768dE32b0b80b7a3454c06BdAc94A69D

Dc7A9

DEBUG_LINK

SUPPLY



protocol:

Aave V3

DEBUG_LINK

(11 OLED frames captured, showing 3)

✓V11 test_advanced_mode_gate

AdvancedMode blind-sign gate (test_msg_ethereum_clear_signing.py)

AdvancedMode OFF + unknown contract + no metadata is hard-rejected; ON signs; a natively-decoded ERC-20 transfer is unaffected.

TRANSACTION



Send the tokens from your wallet, paying

up to 0.0016 ETH for gas?

DEBUG_LINK

SEND



Send 0.01 CVC to

0xd8dA6BF26964aF9D7eEd9e03E53415D37aA9

6045

DEBUG_LINK

(12 OLED frames captured, showing 2)

✓V12 test_cancel_clears_metadata_not_reused

Cancel clears metadata (no stale reuse) (test_msg_ethereum_clear_signing.py)

Cancelling the verified confirm clears the blob; a later matching tx is not silently signed with the stale metadata.

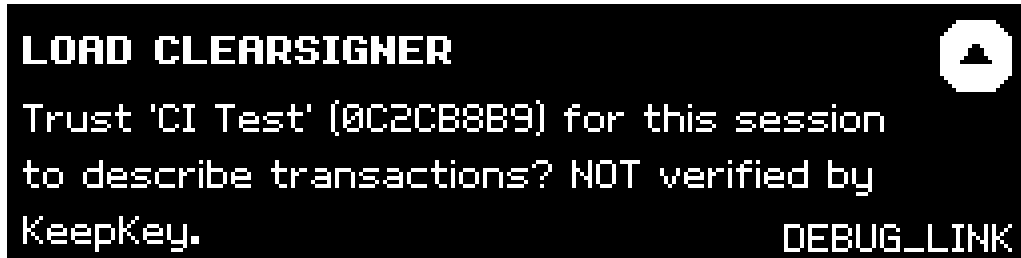
✓V13 test_load_required_before_verify

No built-in key: load required (phase 1) (test_msg_ethereum_clear_signing.py)

On a fresh device a valid metadata blob is MALFORMED until a signer is loaded. Proves there is no hardcoded warning-free trust path in phase 1.

✓V14 test_load_signer_cancel_refuses

Load signer requires on-device consent (test_msg_ethereum_clear_signing.py)
Pressing reject on the LoadClearsignSigner confirm refuses the signer; the slot stays empty and metadata for it is MALFORMED.



(2 OLED frames captured, showing 1)

✓V15 test_load_signer_invalid_pubkey_rejected

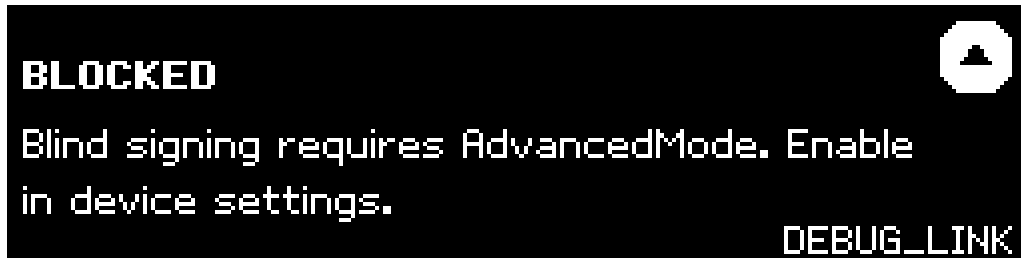
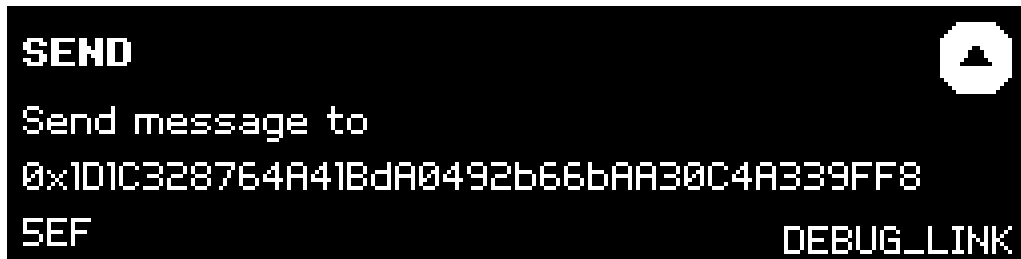
Invalid signer key rejected (test_msg_ethereum_clear_signing.py)
Uncompressed, zero (empty-slot sentinel), and truncated pubkeys are refused before any confirm
- a malicious host cannot install a bogus key.

✓V16 test_load_signer_bad_alias_rejected

Signer alias sanitized (test_msg_ethereum_clear_signing.py)
Empty, oversized, control-char and format-specifier aliases are rejected - the alias is rendered on the warning screen, so it cannot carry a display-spoofing payload.

✓VG1 test_ethereum_blind_sign_blocked

Blind sign refused (AdvancedMode OFF) (test_msg_ethereum_signtx.py)
Unknown contract data with AdvancedMode disabled is hard-rejected before any confirm screen - the negative half of the V8 policy pair.



(4 OLED frames captured, showing 2)

✓VG2 test_eip1559_requires_chain_id

EIP-1559 requires chain_id (test_msg_ethereum_signing_guards.py)
A type-2 tx with no chain_id would hash a garbage pre-image and recover the wrong signer; the device rejects it outright instead of signing an unbroadcastable tx.

✓VG3 test_eip1559_no_priority_fee_signs

EIP-1559 zero priority fee signs correctly (test_msg_ethereum_signing_guards.py)
Regression test for the non-canonical-RLP wrong-signer bug: a type-2 tx with zero/absent priority fee must still hash and sign to the correct device address.

✓VG4 test_type2_without_max_fee_rejected

Type-2 tx without max_fee_per_gas rejected (test_msg_ethereum_signing_guards.py)

✓VG5 test_legacy_with_max_fee_rejected

Legacy tx with max_fee_per_gas rejected (test_msg_ethereum_signing_guards.py)
Mixing legacy gas_price semantics with EIP-1559 fee fields is refused rather than silently mis-hashed.

✓VG6 test_contract_handler_streamed_calldata_signs_full_data

Streamed calldata signs the full payload (test_msg_ethereum_signing_guards.py)
A contract-clear-sign handler must not confirm only the first chunk while signing unshown streamed bytes after it.

✓V17 test_clearsign_erc20_transfer

ERC-20 transfer - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

ERC-20.transfer (core-tokens). The most common on-chain action: send tokens to an address.

AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xa0b8..eb48, chainId 1. Decode: token: USD Coin; to: 0x742d..c8e7; amount: 1 USDC.

SEND



Send 1 USDC to

0x742D35Cc6634C0532950A20547b231011e30C
8e7

DEBUG_LINK

CLEARSIGN



Call:
transfer

DEBUG_LINK

TRANSFER



Contract:

0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606e
B48

DEBUG_LINK

(8 OLED frames captured, showing 3)

✓V18 test_clearsign_erc20_approve

ERC-20 approve - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

ERC-20.approve (approvals). Grants a contract permission to move up to this amount of your tokens. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xa0b8..eb48, chainId 1. Decode: spender: 0x68b3..fc45; amount: 1000 USDC.

APPROVE



Approve withdrawal of up to 1000 USDC by
0x68b3465833fb72A70ecDF485E0e4C7bD866
5Fc45?

DEBUG_LINK

APPROVE



Contract:

0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606e
B48

DEBUG_LINK

APPROVE



spender:

0x68b3465833fb72A70ecDF485E0e4C7bD866
5Fc45

DEBUG_LINK

✓19 test_clearsign_erc20_approve_unlimited

ERC-20 approve - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

ERC-20.approve (approvals). The single most drainer-abused action in EVM: max.uint256 approval.

Must render as "UNLIMITED", never as a raw 78-digit number or hex. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xa0b8..eb48, chainId 1.

Decode: spender: 0x68b3..fc45; amount: UNLIMITED USDC.

IDENTITY	
CI Test (0C2CB8B9)	
describes this tx.	DEBUG_LINK
CLEARSIGN	
Call:	
approve	DEBUG_LINK
APPROVE	
Contract:	
0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606e	
B48	DEBUG_LINK
APPROVE	
spender:	
0x68b3465833fb72A70ecDF485E0e4C7bD866	
5Fc45	DEBUG_LINK
APPROVE	
amount:	
UNLIMITED USDC	DEBUG_LINK
APPROVE	
Unlock full USDC balance for withdrawal by	
0x68b3465833fb72A70ecDF485E0e4C7bD866	
5Fc45?	DEBUG_LINK



TRANSACTION

Send the message from your wallet, paying
up to 0.005 ETH for gas?

DEBUG_LINK

(7 OLED review screens, in order)

✓V20 test_clearsign_uniswap_v2_eth_to_token

Uniswap V2.swapExactETHForTokens - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

Uniswap V2.swapExactETHForTokens (dex-swaps). Swap ETH for a token; the tx VALUE leaving the

wallet is real and shown on the final gas-confirm screen, not hidden in calldata. AdvancedMode

OFF; the bound metadata is the only reason this contract data may sign. Real tx:

to=0x7a25..488d, chainId 1. Decode: protocol: Uniswap V2; amountOutMin: 9.5 USDC; to:

0x742d..c8e7.



IDENTITY

CI Test (0C2CB8B9)
describes this tx.

DEBUG_LINK



CLEARSIGN

Call:

swapExactETHForTokens

DEBUG_LINK



SWAPEXACTETHFORTOKENS

Contract:

0x7a250d5630B4cF539739dF2C5dAcb4c659F
2488D

DEBUG_LINK



SWAPEXACTETHFORTOKENS

protocol:

Uniswap V2

DEBUG_LINK



SWAPEXACTETHFORTOKENS

amountOutMin:

9.5 USDC

DEBUG_LINK

SWAPEXACTETHFORTOKENS



to:

0x742035Cc6634C0532950A20547b231011e30C

8e7

[DEBUG_LINK](#)

SEND



Send 0.01 ETH to

0x7a250d5630B4cF539739dF2C5dAcb4c659F

2488D

[DEBUG_LINK](#)

CONFIRM ETHEREUM DATA



7FF36AB500000000000000000000000000000000

00000000000000000... 228 bytes

[DEBUG_LINK](#)

TRANSACTION



Send 0.01 ETH from your wallet, paying up
to 0.005 ETH for gas?

[DEBUG_LINK](#)

(9 OLED review screens, in order)

✓V21 test_clearsign_uniswap_v2_token_to_eth

Uniswap V2.swapExactTokensForETH - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

Uniswap V2.swapExactTokensForETH (dex-swaps). Both legs of a swap (token in, ETH min-out) shown

in human units. AdvancedMode OFF; the bound metadata is the only reason this contract data may

sign. Real tx: to=0x7a25..488d, chainId 1. Decode: protocol: Uniswap V2; amountIn: 100 USDC;

amountOutMin: 0.003 ETH.

CONFIRM ETHEREUM DATA



18CBAFE500000000000000000000000000000000

00000000000000000... 260 bytes

[DEBUG_LINK](#)

CLEARSIGN



Call:

swapExactTokensForETH

[DEBUG_LINK](#)

SWAPEXACTTOKENSF0RETH



Contract:

0x7a250d5630B4cF539739dF2C5dAcb4c659F

24880

[DEBUG_LINK](#)

(10 OLED frames captured, showing 3)

✓V22 test_clearsign_uniswap_v3_exact_input

Uniswap V3 exactInputSingle - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

Uniswap V3.exactInputSingle (dex-swaps). V3 single-hop swap with an explicit fee tier; typed in/out amounts. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xe592...1564, chainId 1. Decode: protocol: Uniswap V3; tokenIn: 0xc02a..6cc2; tokenOut: 0xa0b8..eb48.

CONFIRM ETHEREUM DATA



414BF389000000000000000000000000000000

C02AAA39B223FE8D... 260 bytes

[DEBUG_LINK](#)

CLEARSIGN



Call:

exactInputSingle

[DEBUG_LINK](#)

EXACTINPUTSINGLE



Contract:

0xE592427A0AEce92De3Edee1F18E0157C05861

564

[DEBUG_LINK](#)

(11 OLED frames captured, showing 3)

✓V23 test_clearsign_uniswap_v3_multicall

Uniswap V3 multicall - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

Uniswap V3.multicall (dex-swaps). Batched calls are opaque by nature; the decode still names the protocol and summarizes in words instead of showing raw bytes[]. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x68b3..fc45, chainId 1. Decode: protocol: Uniswap V3; calls: 1 inner call: refundETH.

CONFIRM ETHEREUM DATA



5AE401DC000000000000000000000000000000

000000000000000000... 196 bytes

[DEBUG_LINK](#)

DEBUG_LINK

DEBUG_LINK

(8 OLED frames captured, showing 3)

✓V24 test clearsign aave v3 pool borrow

Aave V3 borrow - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
 Aave V3.borrow (lending). Draws down a variable-rate loan against posted collateral; onBehalfOf lets a delegator drain credit. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x8787...a4e2, chainId 1. Decode: asset: 0xa0b8...eb48; amount: 1000 USDC; interestRateMode: rate mode: Variable.

DEBUG_LINK

DEBUG_LINK

DEBUG_LINK

(10 OLED frames captured, showing 3)

✓V25 test clearsign aave v3 pool repay

Aave V3 repay - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Aave V3.repay (lending). Pays down outstanding debt; onBehalfOf can pay off someone else's loan. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: 0x0e8787..a4e2, chainId 1. Decode: asset: 0xa0b8..eb48; amount: 500 USDC; interestRateMode: rate mode: Variable.


```
A0B86991C6218B36... 132 bytes
```



Call:
replay



fA4E2

(10 OLED frames captured, showing 3)

Aave V3.withdraw (lending). Redeems supplied collateral for the underlying asset; the classic drainer pattern is a spoofed "to". AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x87877..a4e2, chainId 1. Decode: asset: 0xc02a..6cc2; amount: 2 WETH; to: 0x1234..5678.


```
C02AAA39B223FE8D...    100 bytes
```



fA4E2



56Cc2

(9 OLED frames captured, showing 3)

✓V27 test_clearsign_compound_v3_comet_supply

Compound V3 (Comet) supply - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Compound V3 (Comet).supply (lending). Deposits the base asset into the USDC Comet market to
earn yield or back borrows. AdvancedMode OFF; the bound metadata is the only reason this
contract data may sign. Real tx: to=0xc3d6..cdc3, chainId 1. Decode: asset: 0xa0b8..eb48;
amount: 1000 USDC; protocol: Compound V3 Comet.

SUPPLY

protocol:
Compound V3 Comet

DEBUG_LINK

SUPPLY

Contract:
0xc3d688B66703497DAA19211EEdff47f25384c
dc3

DEBUG_LINK

SUPPLY

asset:
0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606e
B48

DEBUG_LINK

(9 OLED frames captured, showing 3)

✓V28 test_clearsign_compound_v3_comet_withdraw

Compound V3 (Comet) withdraw - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Compound V3 (Comet).withdraw (lending). Withdraws supplied collateral or base-asset balance
from the caller's own Comet account. AdvancedMode OFF; the bound metadata is the only reason
this contract data may sign. Real tx: to=0xc3d6..cdc3, chainId 1. Decode: asset: 0xc02a..6cc2;
amount: 1 WETH; protocol: Compound V3 Comet.

CONFIRM ETHEREUM DATA

F3FEF3A300000000000000000000000000000000
C02AAAB39B223FE8D... 68 bytes

DEBUG_LINK

WITHDRAW

Contract:
0xc3d688B66703497DAA19211EEdff47f25384c
dc3

DEBUG_LINK

56Cc2

(9 OLED frames captured, showing 3)

Spark Protocol supply - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Spark Protocol.supply (lending). Spark is a permissioned Aave V3 fork run by the Sky/MakerDAO ecosystem, sharing Aave's Pool ABI. AdvancedMode OFF; the bound metadata is the only reason this contract data may be on-chain. Real tx: to=0xc13e..e987, chainId 1. Decode: asset: 0x6b17..1d0f; amount: 5000 DAI; onBehalfOf: 0x1234..5678.

E987



E987



5000 DAI

DEBUG_LINK

(10 OLED frames captured, showing 3)

Lido submit - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
 Lido.submit(staking). User stakes ETH directly with Lido's stETH contract and is minted stETH
 1.1. AdvancedCode OFF; the bound metadata is the only reason this contract data may sign. Real
 tx: to=0xae7a..fe84, chainid 1. Decode: referral: 0x0000..0000: action: Lido stETH stake.

```
00000000000000000000..
```

36 bytes

DEBUG_LINK

CLEARSIGN

Call:
submit

DEBUG_LINK

SUBMIT

Contract:
0xae7ab96520DE3A18E5e111B5EaAb09531207fE
84

DEBUG_LINK

(8 OLED frames captured, showing 3)

✓31 test_clearsign_rocketpool_deposit_pool_deposit

Rocket Pool deposit - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Rocket Pool.deposit (staking). User deposits ETH into Rocket Pool's deposit pool and is minted rETH at the current exchange rate. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xdd3f..5af8, chainId 1. Decode: value: 1 ETH; action: Rocket Pool deposit.

DEPOSIT

Contract:
0xDD3f50F8A6CafbE9b31a427582963f465E74
5AF8

DEBUG_LINK

DEPOSIT

value:
1 ETH

DEBUG_LINK

DEPOSIT

action:
Rocket Pool deposit

DEBUG_LINK

(8 OLED frames captured, showing 3)

✓32 test_clearsign_etherfi_liquiditypool_deposit

ether.fi deposit - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
ether.fi.deposit (staking). User deposits ETH into ether.fi's LiquidityPool and is minted rebasing eETH 1:1 in value. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x3088..f216, chainId 1. Decode: _referral: 0x0000..0000; value: 1 ETH; action: ether.fi stake.


```
000000000000000000... 36 bytes
```

DEBUG_LINK

DEBUG_LINK

0x93c4..564d; token: 0xc02a..6cc2; amount: 1 WETH.

DEBUG_LINK

DEBUG_LINK

DEBUG_LINK

(10 OLED frames captured, showing 3)

✓W34 test clearsign eigenlayer strategymanager deposit steth

EigenLayer depositIntoStrategy - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
EigenLayer.depositIntoStrategy (restaking). Same StrategyManager entry point, restaking stETH - the most common real-world case. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x8586..ce72, chainId 1. Decode: strategy: 0x93c4..564d; token: 0xae7a..fe84; amount: 2 stETH.

DEPOSITINTOSTRATEGY



token:

0xae73b96520DE3A18E5e111B5E3Ab095312D7fE

84

DEBUG_LINK

CLEARSIGN



Call:

depositIntoStrategy

DEBUG_LINK

DEPOSITINTOSTRATEGY



Contract:

0x858646372cc42e1Ab8f579c244c0AE3F90cB

CE72

DEBUG_LINK

(10 OLED frames captured, showing 3)

✓35 test clearsign erc20 usdc increase allowance

ERC-20 (USDC) increaseAllowance - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
 ERC-20 (USDC).increaseAllowance (approvals). The front-running-safe alternative to approve() - still grants real spending power. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xa0b8...eb48, chainId 1. Decode: spender: 0x3fc9...7fad; addedValue: 1000000 USDC.

CONFIRM ETHEREUM DATA



39509351000000000000000000000000

3FC91A3AFD70395C...

68 bytes

DEBUG_LINK

CLEARSIGN



Call:

increaseAllowance

DEBUG_LINK

INCREASEALLOWANCE



Contract:

0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606e

B48

DEBUG_LINK

(8 OLED frames captured, showing 3)

✓36 test_clearsign_erc20_usdc_decrease_allowance

ERC-20 (USDC) decreaseAllowance - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

ERC-20 (USDC).decreaseAllowance (approvals). Revocation counterpart to

approve/increaseAllowance - legitimate when reducing a stale allowance. AdvancedMode OFF; the

bound metadata is the only reason this contract data may sign. Real tx: to=0xa0b8..eb48,

chainId 1. Decode: spender: 0x3fc9..7fad; subtractedValue: 500000 USDC.

CONFIRM ETHEREUM DATA



A457C2D700000000000000000000000000000000

3FC91A3AFD70395C...

68 bytes

DEBUG_LINK

CLEARSIGN



Call:

decreaseAllowance

DEBUG_LINK

DECREASEALLOWANCE



Contract:

0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606e

B48

DEBUG_LINK

(8 OLED frames captured, showing 3)

✓37 test_clearsign_eip2612_usdc_permit

ERC-20 (USDC, EIP-2612) permit - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

ERC-20 (USDC, EIP-2612).permit (approvals). The #1 wallet-drainer vector in production: an

off-chain gasless approval, no on-chain fee gate. AdvancedMode OFF; the bound metadata is the

only reason this contract data may sign. Real tx: to=0xa0b8..eb48, chainId 1. Decode: owner:

0x0000..0000; spender: 0x3fc9..7fad; value: UNLIMITED USDC.

CONFIRM ETHEREUM DATA



D505ACCF00000000000000000000000000000000

000000000000000000...

228 bytes

DEBUG_LINK

CLEARSIGN

Call:
permit

DEBUG_LINK

PERMIT

Contract:
0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606e
B48

DEBUG_LINK

(10 OLED frames captured, showing 3)

✓38 test_clearsign_permit2_approve

Uniswap Permit2 approve - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Uniswap Permit2.approve (approvals). Permit2 is a singleton router between the user's ERC-20 allowance and every downstream spender. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x0000..8ba3, chainId 1. Decode: token: 0xa0b8..eb48; spender: 0x3fc9..7fad; amount: UNLIMITED USDC.

CONFIRM ETHEREUM DATA

87517C4500000000000000000000000000000000
A0B86991C6218B36... 132 bytes

DEBUG_LINK

APPROVE

Contract:
0x00000000000022D473030F116dDEE9F6B43aC7
8BA3

DEBUG_LINK

APPROVE

token:
0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606e
B48

DEBUG_LINK

(10 OLED frames captured, showing 3)

✓39 test_clearsign_erc721_bayc_set_approval_for_all

Bored Ape Yacht Club setApprovalForAll - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Bored Ape Yacht Club.setApprovalForAll (approvals). Grants an operator blanket control over EVERY token the owner holds in this collection. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xbc4c..f13d, chainId 1. Decode: operator: 0x3fc9..7fad; approved: grants control of ALL NFTs; collection: Bored Ape Yacht Club.

SETAPPROVALFORALL



Contract:

0xBC4CA0EdA7647A8aB7C2061c2E118A18a936f1

3D

[DEBUG_LINK](#)

CLEARSIGN



Call:

setApprovalForAll

[DEBUG_LINK](#)

SETAPPROVALFORALL



operator:

0x3fC91A3afd70395Cd496C647d5a6CC9D4B2b

7FAD

[DEBUG_LINK](#)

(9 OLED frames captured, showing 3)

✓V40 test_clearsign_erc1155_opensea_storefront_set_approval_for_all

OpenSea Shared Storefront setApprovalForAll - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

OpenSea Shared Storefront.setApprovalForAll (approvals). Identical blanket-operator risk to ERC-721, on a shared ERC-1155 storefront contract. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x495f..7b5e, chainId 1. Decode: operator: 0x3fc9..7fad; approved: grants control of ALL items; collection: OpenSea Storefront.

SETAPPROVALFORALL



Contract:

0x495f947276749Ce646f68AC8c248420045c

b7b5e

[DEBUG_LINK](#)

CLEARSIGN



Call:

setApprovalForAll

[DEBUG_LINK](#)

SETAPPROVALFORALL



operator:

0x3fC91A3afd70395Cd496C647d5a6CC9D4B2b

7FAD

[DEBUG_LINK](#)

(9 OLED frames captured, showing 3)

✓41 test_clearsign_usdt_approve

ERC-20 (USDT) approve - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
 ERC-20 (USDT).approve (approvals). USDT's approve() omits the standard non-zero-to-non-zero guard other tokens have. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xdac1...1ec7, chainId 1. Decode: spender: 0x3fc9..7fad; amount: 500 USDT.

APPROVE 

Approve withdrawal of up to 500 USDT by
0x3fc91A3afd70395Cd496C647d5a6CC9D4B2b
7FAD? DEBUG_LINK

APPROVE 
Contract:
0xdAC17F958D2ee523a2206206994597C13D831
ec7 DEBUG_LINK

APPROVE 

spender:
0x3fC91A3afd70395Cd496C647d5a6CC9D4B2b
7FAD [DEBUG_LINK](#)

(7 OLED frames captured, showing 3)

✓V42 test_clearsign_dai_permit

Dai Stablecoin permit - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
 Dai Stablecoin.permit (approvals). DAI's permit is boolean allowed/not-allowed, not a partial amount - a subtle drainer trap if a wallet renders it like a normal EIP-2612 permit.
 AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx:
 to=0x6b17...1d0f, chainId 1. Decode: holder: 0x286c...1d60; spender: 0x3fc9...7fad; allowed:
 grant: unlimited allowance.

CONFIRM ETHEREUM DATA 

8FCBAF0C00

28C6C06298D514DB... 260 bytes

[DEBUG_LINK](#)

CLEARSIGN  Call: permit [DEBUG_LINK](#)


```
0x6B175474E89094C440a98b954EedeAC49527
1d0F                                DEBUIG
```

DEBUG_LINK

✓V43 test_clearsign_erc721_safe_transfer_from

ERC-721 (BAYC) safeTransferFrom - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
ERC-721 (BAYC).safeTransferFrom (nft-transfer). Direct peer-to-peer ERC-721 transfer with no on-chain price/consideration. AdvancedModel Code OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xbc4c.f13d, chainId 1. Decode: from: 0x7a16..542a; to: 0x742d..c8e7; tokenId: NFT: BAYC #4576.

[illegible]

DEBUG_LINK

safeTransferFrom

DEBUG_LINK

0xBC4CA0EdA7647A8aB7C2061c2E118A18a936f1
3D DEB11G

DEBUG_LINK

✓44 test_clearsign_safe_addownerwiththreshold

Safe (Gnosis Safe) addOwnerWithThreshold - clear-signed, zero hex (test_msg_ethereum_clear_signing.py) Safe (Gnosis Safe).addOwnerWithThreshold (account-abstraction). Only reachable self-referentially inside a Safe's own execTransaction - a malicious co-signer could try to add an attacker-controlled owner and lower the threshold to seize the Safe. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x1b9c..81b9, chainId 1. Decode: owner: 0x0000..beef; _threshold: new threshold: 3 owners; protocol: Safe: governance change.

[illegible]

DEBUG_LINK

addOwnerWithThreshold

DEBUG_LINK

0x1B9Cef6Bdd029f378c511E5e6C20eE556b678

169

DEBUG_LINK

(9 OLED frames captured, showing 3)

✓V45 test clearsign hop protocol l1 bridge sendtol2

Hop Protocol sendToL2 - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

Hop Protocol.sendToL2 (bridge). Deposits into Hop's L1 AMM/bridge; a bondor fronts liquidity on the destination chain. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x3666e..f18a, chainId 1. Decode: chainId: destination: Polygon; recipient: 0x742d..c8e7; amount: 250 USDC.

[illegible]

```
00000000000000000000... 228 bytes
```

DEBUG_LINK

sendToL2

DEBUG_LINK

0x3666f603C164936C1b87e207F36BEBa4AC5

f18a

DEBUG_LINK

(10 OLED frames captured, showing 3)

✓V46 test clearsign wormhole token bridge transfertokens

Wormhole transferTokens - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

Wormhole.transferTokens(bridge). Locks the ERC-20 in Token Bridge custody and emits a message Wormhole's guardians attest to. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: `to=0x3ee1..a585, chainId 1. Decode: token: 0xa0b8..eb48; amount: 100 USD; recipientChain: dest: Arbitrum (Wormhole).`

DEBUG_LINK

DEBUG_LINK

DEBUG_LINK

ID: 203; support: 0=Against 1=For 2=Abstain.

DEBUG_LINK

DEBUG_LINK

DEBUG | TNK

(8 OLED frames captured, showing 3)

✓48 test_clearsign_ens_public_resolver_setaddr

ENS setAddr - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
ENS.setAddr (governance). Updates the ETH address a .eth name resolves to; callable only by the name's controller. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x231b..8e63, chainId 1. Decode: node: ENS name (namehash); a: 0xd8da..6045.

CONFIRM ETHEREUM DATA

D5FA2B00EE6C4522AAB0003E8D14CD40
A6AF439055FD2577... 68 bytes

DEBUG_LINK

CLEARSIGN

Call:
setAddr

DEBUG_LINK

SETADDR

Contract:
0x231b0Ee14048e9dCcD1d247744d114a4EB5E8E
63

DEBUG_LINK

(8 OLED frames captured, showing 3)

✓49 test_clearsign_metamorpho_steakhouse_usdc_deposit

Morpho (Steakhouse USDC) deposit - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Morpho (Steakhouse USDC).deposit (vaults). Standard ERC-4626 deposit into a MetaMorpho vault built on Morpho Blue. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xbeef..64cb, chainId 1. Decode: assets: 1000 USDC; receiver: 0x0000..beef; protocol: Steakhouse USDC vault.

DEPOSIT

Contract:
0xBEEF01735c132AdA46AA9aA4c54623cAA92A
64CB

DEBUG_LINK

DEPOSIT

assets:
1000 USDC

DEBUG_LINK

DEPOSIT



Contract:

0x5f18C75AbDAe578b483E5F43f12a39cF75b9
73a9

DEBUG_LINK

DEPOSIT



_amount:

1000 USDC

DEBUG_LINK

(8 OLED frames captured, showing 3)

✓V52 test_clearsign_yearn_v3_aave_usdc_lender_deposit

Yearn Finance (V3) deposit - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Yearn Finance (V3).deposit (vaults). Yearn V3's tokenized-strategy ERC-4626 vault passes
deposits through to Aave V3. AdvancedMode OFF; the bound metadata is the only reason this
contract data may sign. Real tx: to=0xbdb9..7be4, chainId 1. Decode: assets: 1000 USDC;
receiver: 0x0000..beef; protocol: Yearn V3 Aave USDC.

SEND



Send message to

0xbDb97eC319c41c6FA383E94eCE6Bdf383dFC
7BE4

DEBUG_LINK

DEPOSIT



Contract:

0xbDb97eC319c41c6FA383E94eCE6Bdf383dFC
7BE4

DEBUG_LINK

DEPOSIT



assets:

1000 USDC

DEBUG_LINK

(9 OLED frames captured, showing 3)

✓V53 test_clearsign_compound_iii_comet_usdc_supply

Compound III (Comet) supply - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Compound III (Comet).supply (vaults). Supplying USDC as the Comet base asset mints a rebasing
cUSDCv3 balance earning yield. AdvancedMode OFF; the bound metadata is the only reason this
contract data may sign. Real tx: to=0xc3d6..cdc3, chainId 1. Decode: asset: 0xa0b8..eb48;
amount: 1000 USDC; protocol: Compound III Comet.

SUPPLY

protocol:
Compound III Comet

DEBUG_LINK

SUPPLY

Contract:
0xc3d688B66703497DAA19211EEdff47f25384c
dc3

DEBUG_LINK

SUPPLY

asset:
0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606e
B48

DEBUG_LINK

(9 OLED frames captured, showing 3)

✓V54 test_clearsign_weth_deposit

WETH9 deposit - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

WETH9.deposit (core-tokens). deposit() takes no calldata; the ETH being wrapped is carried entirely in the tx value. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xc02a..6cc2, chainId 1. Decode: action: Wrap ETH into WETH; amount: 1 ETH.

DEPOSIT

Contract:
0xC02aaA39b223FE8D0A0e5C4F27eAD9083C7
56Cc2

DEBUG_LINK

DEPOSIT

action:
Wrap ETH into WETH

DEBUG_LINK

DEPOSIT

amount:
1 ETH

DEBUG_LINK

(8 OLED frames captured, showing 3)

WETH9 withdraw - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
WETH9.withdraw(core-tokens). Burns wad WETH from the caller and sends wad ETH back to msg.sender. AdvancedToken OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xc02a...6cc2. chainId 1. Decode: wad: 0.5 WETH.

WITHDRAW 

wad:
0.5 WETH

[DEBUG_LINK](#)

(7 OLED frames captured, showing 3)

ERC-20 (USDT) transferFrom - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
 ERC-20 (USDT).transferFrom (core-tokens). The highest-risk ERC-20 call for a hardware wallet to
 sign: the signer (msg.sender/spender) moves funds OUT of a DIFFERENT account (from) that
 pre-approved it - "from" is not the signer. AdvancedMode OFF; the bound metadata is the only
 reason this contract data may sign. Real tx: to=0xdac1...1ec7, chainId 1. Decode: action: pull
 from approved account; from: 0x7a1b6...542a; to: 0x742d...cbe7.

CLEARSIGN 
Call:
transferFrom
DEBUG_LINK

TRANSFERFROM



Contract:

0xdAC17F958D2ee523a2206206994597C13D831

ec7

DEBUG_LINK

(10 OLED frames captured, showing 3)

✓V57 test_clearsign_uniswap_v3_exact_output_single

Uniswap V3 exactOutputSingle - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Uniswap V3.exactOutputSingle (dex-swaps). Reverse-direction swap (buy an exact output instead of spending an exact input) - the risk is amountInMax, an implicit "pay up to" ceiling.
AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x68b3..fc45, chainId 1. Decode: protocol: Uniswap V3; tokenIn: 0xa0b8..eb48; tokenOut: 0xc02a..6cc2.

CONFIRM ETHEREUM DATA



5023B4DF000000000000000000000000

A0B86991C6218B36... 228 bytes

DEBUG_LINK

CLEARSIGN



Call:

exactOutputSingle

DEBUG_LINK

EXACTOUTPUTSINGLE



Contract:

0x68b3465833fb72A70ecDF485E0e4C7bD866

5Fc45

DEBUG_LINK

(12 OLED frames captured, showing 3)

✓V58 test_clearsign_curve_3pool_exchange

Curve Finance (3pool) exchange - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Curve Finance (3pool).exchange (dex-swaps). 3pool coin indices (0=DAI,1=USDC,2=USDT) are fixed but not self-describing on-chain - a hardware wallet must translate the index to a coin name, not show a bare "1". AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0xbebc..f1c7, chainId 1. Decode: i: sell coin index: 1 (USDC); j: buy coin index: 2 (USDT); dx: 1000 USDC.

CONFIRM ETHEREUM DATA



30F02124000000000000000000000000

000000000000000000... 132 bytes

DEBUG_LINK

DEBUG_LINK

DEBUG_LINK

(10 OLED frames captured, showing 3)

✓V59 test clearsign erc1155 safe transfer from

ERC-1155 safeTransferFrom - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
 ERC-1155 safeTransferFrom (nft-transfer). ERC-1155 amount is a raw edition count, not a decimal-scaled token amount - a wallet that runs it through TOKEN_AMOUNT formatting would show a nonsense value. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x495f7b5e, chainId 1. Decode: from: 0x7a16..542a; to: 0x742d..c8e7; tokenId: NFT; OpenSea Storefront item.

DEBUG_LINK

DEBUG_LINK

DEBUG_LINK

(10 OLED frames captured, showing 3)

✓V60 test clearsign erc1155 safe batch transfer from

ERC-1155 safeBatchTransferFrom - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
 ERC-1155 safeBatchTransferFrom (nft-transfer). Atomic batch transfer of multiple ids/quantities
 - a wallet screen can only show a handful of typed fields, so a long batch MUST be summarized,
 never left as raw arrays. AdvancedMode OFF; the bound metadata is the only reason this contract
 data may sign. Real tx: to=0x495f7b5e, chainId 1. Decode: from: 0x7a16..542a; to:
 0x742d3c8e7; ids: 2 NFT ids in this batch.

DEBUG_LINK

DEBUG_LINK

DEBUG_LINK

Uniswap V4 execute - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
 Uniswap V4.execute(dex-swaps). V4's command-based router means the swap itself is opaque
 bytes; the decode must at least name the protocol and the command type, not show raw commands
 hex. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real
 tx: to=0x66a9..a8af, chainId 1. Decode: protocol: Uniswap V4 (Universal Router); commands:
 command: 0x10 (V4 SWAP); deadline: expires 2027-12-28 13:20 UTC.

DEBUG_LINK

DEBUG_LINK

DEBUG_LINK

(9 OLED frames captured, showing 3)

✓V62 test_clearsign_permit2_permit_transfer_from

Uniswap Permit2 (SignatureTransfer).permitTransferFrom - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Uniswap Permit2 (SignatureTransfer).permitTransferFrom (approvals). The authorization for this transfer was a PURE off-chain EIP-712 signature made earlier (often on a phishing site) - this call is the moment the funds actually move. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x0000..8ba3, chainId 1. Decode: token: 0xa0b8..eb48; amount: 250000 USDC; recipient: 0x3fc9..7fad.

IDENTITY

CI Test (0C2CB8B9)
describes this tx.

DEBUG_LINK

CLEARSIGN

Call:
permitTransferFrom

DEBUG_LINK

PERMITTRANSFERFROM

Contract:
0x000000000022D473030F116dDEE9F6B43aC7
8BA3

DEBUG_LINK

PERMITTRANSFERFROM

token:
0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606e
B48

DEBUG_LINK

PERMITTRANSFERFROM

amount:
250000 USDC

DEBUG_LINK

PERMITTRANSFERFROM

recipient:
0x3fc91A3afd70395Cd496C647d5a6CC9D4B2b
7FAD

DEBUG_LINK

PERMITTRANSFERFROM

deadline:
expires 2027-12-28 13:20 UTC

DEBUG_LINK

SEND

Send message to
0x00000000000022D473030F116dDEE9F6B43aC7
8BA3

DEBUG_LINK

CONFIRM ETHEREUM DATA

30F28B7A000000000000000000000000
A0B86991C6218B36... 388 bytes

DEBUG_LINK

TRANSACTION

Send the message from your wallet, paying
up to 0.005 ETH for gas?

DEBUG_LINK

(10 OLED review screens, in order)

✓63 test_clearsign_across_spokepool_depositv3

Across Protocol depositV3 - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)
Across Protocol.depositV3 (bridge). ERC-7683-style intent bridge: locks the input token so an
unbonded relayer can front the output token on the destination chain - the signature doesn't
show final asset movement, so the decode must make output token/amount/chain explicit.
AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx:
to=0x5c7b..35c5, chainId 1. Decode: inputToken: 0xa0b8..eb48; inputAmount: 1000 USDC;
outputToken: 0xaf88..5831.

CONFIRM ETHEREUM DATA

7B93923200000000000000000000000000
742D35CC6634C053... 420 bytes

DEBUG_LINK

CLEARSIGN

Call:
depositV3

DEBUG_LINK

DEPOSITV3



Contract:

0x5c7BCd6E7De5423a257D81B442095A1a6ced

35C5

[DEBUG_LINK](#)

(11 OLED frames captured, showing 3)

✓64 test_clearsign_safe_exectransaction

Safe (Gnosis Safe) execTransaction - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

Safe (Gnosis Safe).execTransaction (account-abstraction). A co-signing Safe owner signs this

off-chain "Safe transaction hash" with their hardware wallet before relaying; operation=1

(DELEGATECALL) would run arbitrary code as the Safe itself - the single field a wallet must

never let slide by unshown. AdvancedMode OFF; the bound metadata is the only reason this

contract data may sign. Real tx: to=0x1b9c..81b9, chainId 1. Decode: to: 0xa0b8..eb48;

operation: call type: 0=CALL; gasBudget: gas budget: 150000.

IDENTITY



CI Test (0C2CB8B9)

describes this tx.

[DEBUG_LINK](#)

CLEARSIGN



Call:

execTransaction

[DEBUG_LINK](#)

EXECTransaction



Contract:

0x1B9Cef6Bdd029f378c511E5e6C20eE556b678

1b9

[DEBUG_LINK](#)

EXECTransaction



to:

0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606e

B48

[DEBUG_LINK](#)

EXECTransaction



operation:

call type: 0=CALL

[DEBUG_LINK](#)

DEBUG_LINK

DEBUG_LINK

DEBUG_LINK

DEBUG_LINK

DEBUG_LINK

nonce: UserOperation nonce: 12; beneficiary: 0x4343..4343.



IDENTITY

CI Test (0C2CB8B9)
describes this tx.

[DEBUG_LINK](#)



CLEARSIGN

Call:
handleOps

[DEBUG_LINK](#)



HANDLEOPS

Contract:
0x0000000071727De22E5E9d8BAf0edAc6f37d
a032

[DEBUG_LINK](#)



HANDLEOPS

sender:
0x9406Cc6185a346906296840746125a0E4497
6454

[DEBUG_LINK](#)



HANDLEOPS

nonce:
UserOperation nonce: 12

[DEBUG_LINK](#)



HANDLEOPS

beneficiary:
0x434343434343434343434343434343434
34343

[DEBUG_LINK](#)



HANDLEOPS

innerCall:
empty (representative)

[DEBUG_LINK](#)

SEND



Send message to

0x0000000071727De22E5E9d8BAf0edAc6f37d

a032

[DEBUG_LINK](#)

CONFIRM ETHEREUM DATA



765E827F000000000000000000000000

0000000000000000... 548 bytes

[DEBUG_LINK](#)

TRANSACTION



Send the message from your wallet, paying
up to 0.005 ETH for gas?

[DEBUG_LINK](#)

(10 OLED review screens, in order)

✓V66 test_clearsign_eip7702_setcode_authorization

EIP-7702 (Set Code for EOAs) authorization - clear-signed, zero hex (test_msg_ethereum_clear_signing.py)

EIP-7702 (Set Code for EOAs).authorization (account-abstraction). This EOA is authorizing delegation to a contract - NOT a normal contract call. A malicious 7702 delegation disguised as a routine signature is effectively account takeover; the delegate address must be shown with the same weight as a recipient. AdvancedMode OFF; the bound metadata is the only reason this contract data may sign. Real tx: to=0x4cd2..66c9, chainId 1. Decode: txType: NEW: type-0x04 (EIP-7702); delegate: 0x4cd2..66c9; chainScope: chain 1 only (0 = ALL chains).

IDENTITY



CI Test (0C2CB8B9)

describes this tx.

[DEBUG_LINK](#)

CLEARSIGN



Call:

authorization

[DEBUG_LINK](#)

AUTHORIZATION



Contract:

0x4Cd241E8d1510e30b2076397afc7508Ae59C6

6c9

[DEBUG_LINK](#)



AUTHORIZATION

txType:

NEW: type-0x04 (EIP-7702)

[DEBUG_LINK](#)



AUTHORIZATION

delegate:

0x4Cd241E8d1510e30b2076397afc7508Ae59C6
6c9

[DEBUG_LINK](#)



AUTHORIZATION

chainScope:

chain 1 only (0 = ALL chains)

[DEBUG_LINK](#)



AUTHORIZATION

effect:

EOA becomes alias for this code

[DEBUG_LINK](#)



SEND

Send message to

0x4Cd241E8d1510e30b2076397afc7508Ae59C6
6c9

[DEBUG_LINK](#)



CONFIRM ETHEREUM DATA

04000000

4 bytes

[DEBUG_LINK](#)



TRANSACTION

Send the message from your wallet, paying
up to 0.005 ETH for gas?

[DEBUG_LINK](#)

✓V67 test_clearsign_batch_all_payloads

Batch: sign + device-validate the whole catalog (test_msg_ethereum_clear_signing.py)

Signs every CLEARSIGN_FLOWS payload (51 real-world flows spanning DEX swaps, lending, staking, approvals/permits, NFTs, governance, bridges, and account abstraction - ERC-4337, EIP-7702, Safe multisig, Permit2, Uniswap V4) in one batch and has the device validate each: every blob returns VERIFIED, and the same blob with one tampered byte returns MALFORMED. Together with the frozen offline reference vectors (RFC 6979 deterministic - byte-identical blobs, sha256 snapshots in the test), this makes python-keepkey the complete signer reference: produce these bytes and the device accepts them; deviate by one byte and it refuses.

✓VS1 test_layout_has_no_tx_hash

v2 schema blob carries no tx_hash / no values (test_msg_ethereum_clear_signing.py)

The v2 (static schema) blob attests only how to decode a curated (chainId, contract, selector): method + per-arg name/format (+ static decimals/symbol). It has NO committed tx_hash and NO argument values - so it can be signed ONCE, offline, and served from a CDN with no hot key. The device decodes the values itself from the calldata it signs.

✓VS2 test_token_arg_carries_static_decimals_symbol_not_value

v2 token arg = static decimals/symbol, value decoded on-device (test_msg_ethereum_clear_signing.py)

A TOKEN_AMOUNT arg encodes the token's static decimals + symbol (a property of the contract), but NOT the amount - the amount is decoded from the calldata word on-device, then rendered "1.5 USDC".

✓VS3 test_frozen_body_snapshot

v2 wire format frozen vs firmware parser (test_msg_ethereum_clear_signing.py)

The canonical v2 body's length + sha256 are frozen, so the serializer can never drift from firmware's parse_v2_args() undetected - the same byte-parity discipline the v1 reference vectors use.

✓VS4 test_rejects_dynamic_format

v2 scope: fixed-word types only (test_msg_ethereum_clear_signing.py)

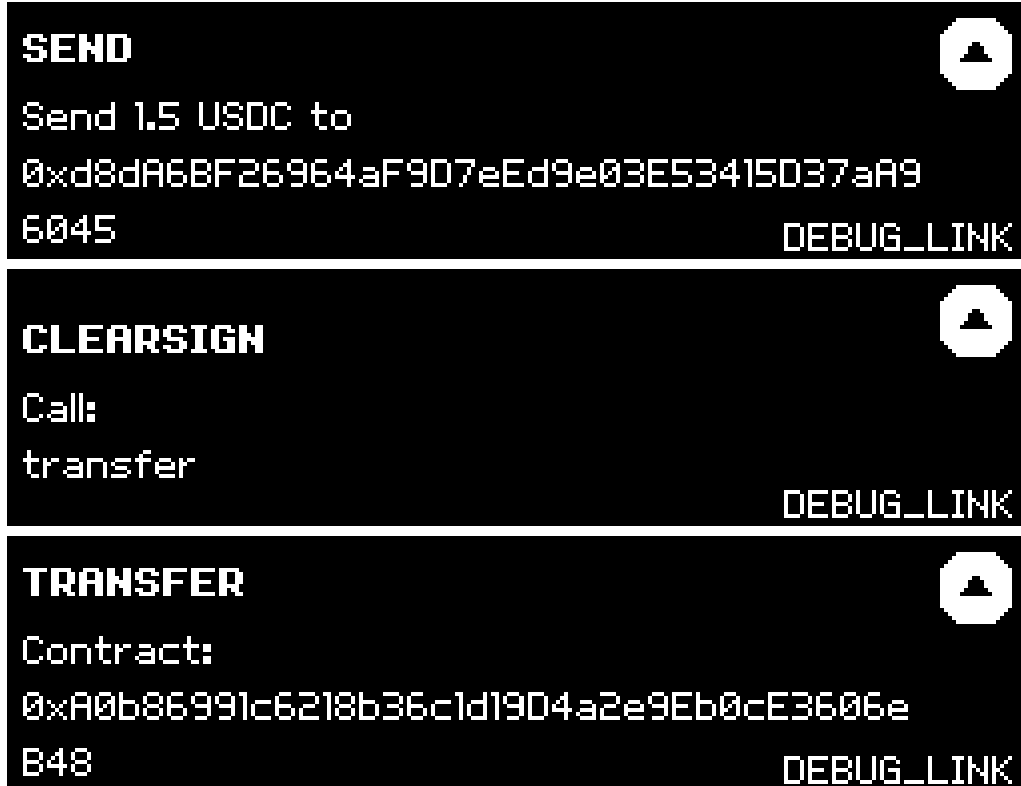
v2 decodes fixed single ABI words (ADDRESS / AMOUNT / TOKEN_AMOUNT) - approve/transfer/transferFrom and fixed-arg calls. Dynamic types (string/bytes/arrays) are rejected by the serializer and fall to the blind-sign path on-device; a bounded dynamic decoder is future work.

✓VS5 test_v2_transfer_decodes_signs_and_recovers

v2 on-device: decode from calldata, sign, recover (test_msg_ethereum_clear_signing.py)

END-TO-END with AdvancedMode OFF: a v2 transfer() schema blob + a real transfer(to, amount) tx.

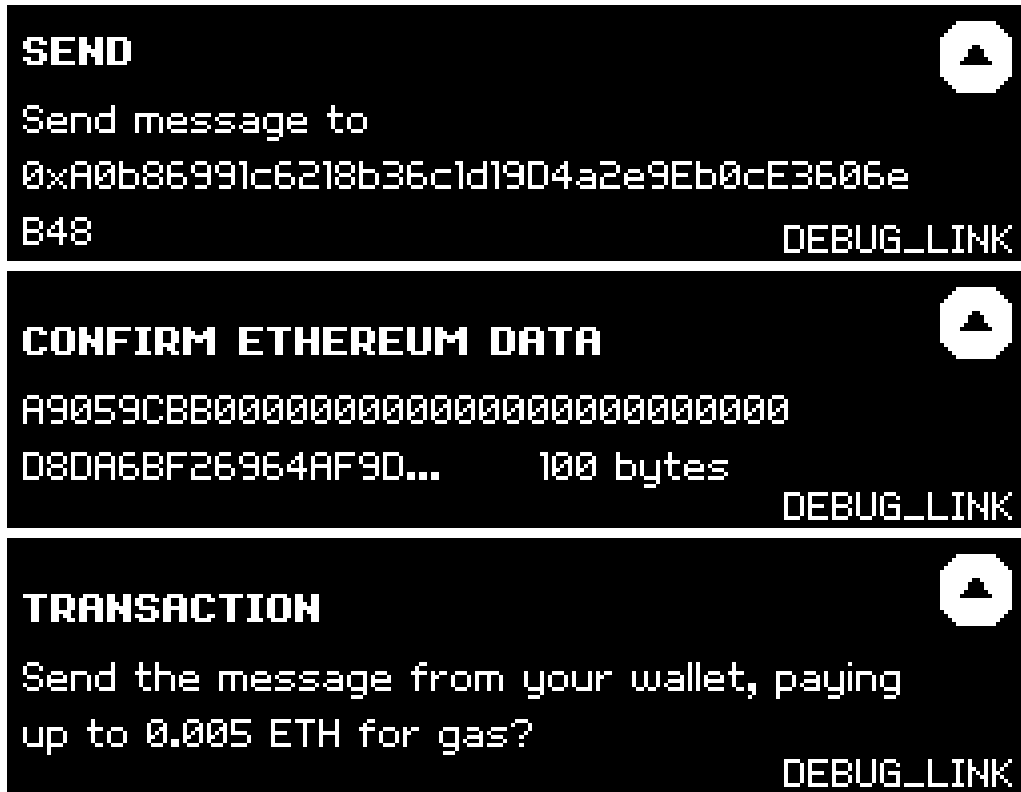
The device decodes to/amount from the calldata and clear-signs; the signature recovers to this device's signer over the tx digest - so the who/what/why shown was bound to the exact tx, with no tx_hash. The offline format tests above pin the wire format the device decodes.



(7 OLED frames captured; showing 3)

✓VS6 test_v2_calldata_length_mismatch_falls_back_to_raw_review

v2 decode-mismatch falls back to raw review (fail-closed) (test_msg_ethereum_clear_signing.py)
 THE headline v2 security property: schema says 2 words, calldata carries 3. decode_v2_args' structural completeness check fails, so the device does NOT clear-sign a decode that would not match what it is about to sign. With AdvancedMode ON it falls through to the ordinary unverified raw review, and the ordered OLED captures prove the decoded ClearSign display was not used.



(3 OLED review screens, in order)

✓VS7 test v2 unsupported_arg format returns malformed

v2 unsupported arg format rejected at blob load (test_msg_ethereum_clear_signing.py)
A hand-crafted v2 blob using an unsupported dynamic format (STRING) - the kind the Python serializer itself refuses to build - is independently rejected by the device's own parser as MALFORMED, before any calldata is even considered.

G. Hive [NEW]

NEW: Hive (Graphene) support with SLIP-0048 role derivation. Four role keys per account (owner, active, posting, memo), each an STM-prefixed secp256k1 key. Signs Graphene transactions - transfer, the account-create / account-update authority operations Pioneer uses to onboard sponsored accounts, Keychain signBuffer message signing (dApp login), and parsed generic operations (vote, comment, custom_json). Every signature recovers to the role key it was signed under, each serialized field is bound at its byte position, and every user-controlled string is paged IN FULL on the OLED (72-char ASCII pages; non-ASCII shown as complete hex). Message signing is restricted to printable ASCII: a Hive transaction digest is SHA256(chain_id || binary tx), so the printable-only whitelist makes signable messages provably disjoint from every transaction preimage on ANY fork chain - closing the message->transaction signature-oracle class.

User Flow

KEYS: SLIP-0048 m/48'/13'/role'/0'/account' -> STM-prefixed pubkey per role
SIGN TX: Graphene serialize -> per-op confirm (amount + recipient + full memo pages) -> ECDSA sign
ACCOUNT CREATE: attest 4 role authorities + new-account name -> owner-key signature
SIGN MESSAGE: printable ASCII only -> role named + full message pagged -> SHA256(msg) signed
SIGN OPS: device re-parses the Graphene bytes; unrecognized ops are refused (no blind-sign)

Tests: 38/38 -- ALL PASSED

✓G1 test hive get public key active

Derive active-role key (test_msg_hive.py)
Active-role key derives and returns an STM-prefixed key plus the 33-byte compressed raw pubkey (0x02/0x03 prefix).

✓G2 test_hive_get_public_keys_all_roles

Derive all four role keys (test_msg_hive.py)

Owner, active, posting and memo keys all derive, are distinct, and STM-formatted. The bulk path agrees with the single-key path for the active role.

✓G3 test_hive_sign_transfer

Sign Hive transfer (test_msg_hive.py)

Transfer (op 2) signs; the signature recovers to the active key. The device shows the recipient account and amount, and every serialized field (from/to/amount/asset/memo) is bound at its position so a rewritten recipient or amount fails.

**MEMO**

kktest

DEBUG_LINK

**SEND HIVE**

Send 1 HIVE to @kkrecipient?

DEBUG_LINK

**SIGN TRANSACTION**

Sign Hive transaction from @kktester?

DEBUG_LINK

✓G4 test_hive_sign_account_create

Sign account-create attestation (test_msg_hive.py)

account_create (op 9) signs and recovers to the owner key - the attestation a Pioneer sponsor verifies before spending an account-creation token. Binds the four role authorities, creator, new-account name and fee at their exact positions.

**CREATE HIVE ACCOUNT**

Create @kktestacct secured by KeepKey?

All keys from your device.

DEBUG_LINK

**OWNER KEY**

STM6D1AabbhJFXQGxaDUB61ecaPzciJ6emE9aKw
MyrqoYnJbygs9X2

DEBUG_LINK



CREATION FEE

Fee: 3.000 HIVE paid by @kksponsor

DEBUG_LINK

✓G5 test_hive_sign_account_update

Sign account-update (test_msg_hive.py)
account_update (op 10) signs and recovers to the owner key; the replacement authorities are bound to their slots so updating the wrong authority fails.



NEW OWNER KEY

STM6D1AabbhJFXQGxaDUB61ecaPzcij6emE9aKw
MyrqoYnJbygs9X2

DEBUG_LINK



SECURE HIVE ACCOUNT 1/2

Replace ALL keys for @kktestacct with
KeepKey keys?

DEBUG_LINK



SECURE HIVE ACCOUNT 2/2

Old keys will be retired.

DEBUG_LINK

✓G6 test_hive_sign_transfer_max_memo_ok

Max-length memo paged in full (boundary) (test_msg_hive.py)
A memo of exactly 440 bytes (the serialization limit) still signs, and the OLED pages the COMPLETE memo (MEMO 1/7..7/7) - nothing is truncated behind a benign-looking prefix.



MEMO 2/4

XX

XX

XX

DEBUG_LINK



SEND HIVE

Send 1 HIVE to @kkrecipient?

DEBUG_LINK

GETTING_LINK

✓G7 test_hive_sign_transfer_rejects_long_memo

A 441-byte memo fails with a specific "memo too long" error before any signing. Rejection happens before any confirm UI, so there is no OLED frame - the proof is the specific device error.

Foreign derivation paths rejected (test_msg_hive.py)

✓G9 test_hive_sign_transfer_rejects_wrong_network

A path whose network index is not Hive (13') must not sign.

Transfer requires the active role (test_msg_hive.py)

✓G11 test_hive_sign_message_posting

Keychain signBuffer contract: signature over SHA256(raw message bytes) with the posting key.

DEBUG_LINK

✓G12 test_hive_sign_message_all_roles

Posting, active and memo roles may sign (owner's is refused); each signature recovers to that role's distinct key.

DEBUG_LINK

DEBUG_LINK



SIGN HIVE MESSAGE

Signing with memo key

DEBUG_LINK

(6 OLED frames captured, showing 3)

✓G13 test_hive_sign_message_long_printable_ok

Long message paged in full (test_msg_hive.py)

Printable text over the display budget routes through 72-char pages - never silently truncated
- and the signature covers every byte.



HIVE MESSAGE 2/3

eamble. benign preamble. benign preamble.
benign preamble. benign preamble. benign
preamble. benign preamble. benign preamb

DEBUG_LINK



HIVE MESSAGE 1/3

benign preamble. benign preamble. benign
preamble. benign preamble. benign preamble.
benign preamble. benign preamble. benign

DEBUG_LINK



HIVE MESSAGE 3/3

benign preamble. benign preamble. benign
prea

DEBUG_LINK

(4 OLED frames captured, showing 3)

✓G14 test_hive_sign_message_max_length_ok

Max-length (1024 B) message (test_msg_hive.py)

A message of exactly 1024 bytes (the proto cap) pages and signs.



HIVE MESSAGE 8/10

XX
XX
XX

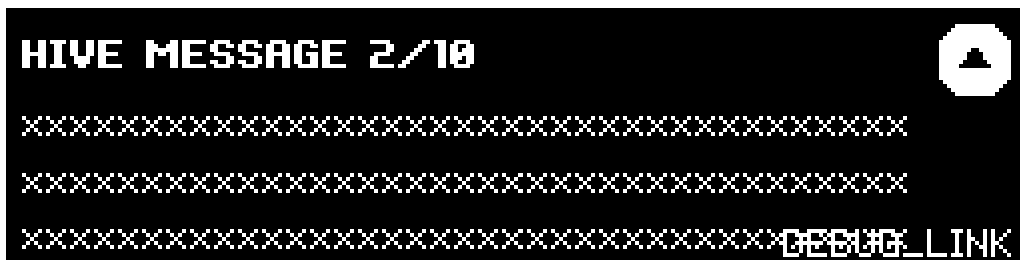
DEBUG_LINK



HIVE MESSAGE 1/10

XX
XX
XX

DEBUG_LINK



(11 OLED frames captured, showing 3)

✓G15 test_hive_sign_message_nonprintable_bytes

SECURITY: binary messages refused (oracle fix) (test_msg_hive.py)
A binary "message" equal to chain_id || serialized_tx would hash to a valid TRANSACTION signature on any fork chain - an active-key fund-theft oracle. The printable-ASCII whitelist refuses every binary buffer, making signable messages provably disjoint from all transaction preimages. Rejection is pre-UI (no frame); the proof is the "printable" device error.

✓G16 test_hive_sign_message_rejects_chain_id_prefix

Chain-id-prefixed message refused (test_msg_hive.py)
Belt-and-suspenders subset of G15: a message starting with the Hive mainnet chain id is refused outright.

✓G17 test_hive_sign_message_rejects_oversize

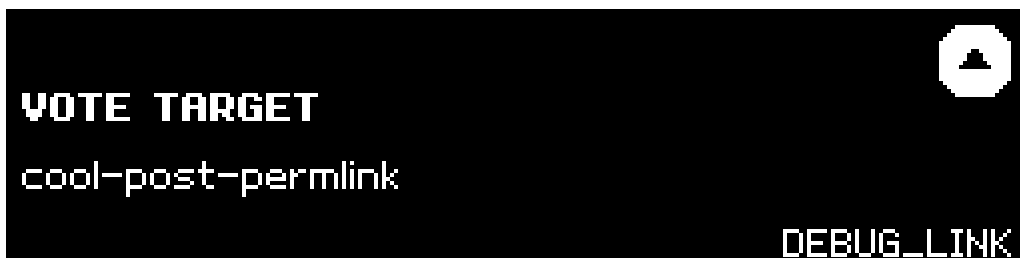
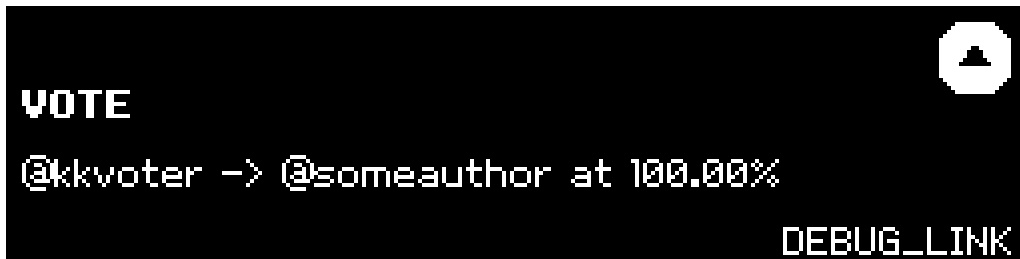
Oversize message refused (test_msg_hive.py)
1025 bytes must fail - the proto cap and the handler agree on 1024.

✓G18 test_hive_sign_message_rejects_bad_paths

Message signing path fence (test_msg_hive.py)
Foreign trees, wrong network, unassigned roles, owner' and short paths are all refused - the same SLIP-0048 fence as transactions.

✓G19 test_hive_sign_ops_vote

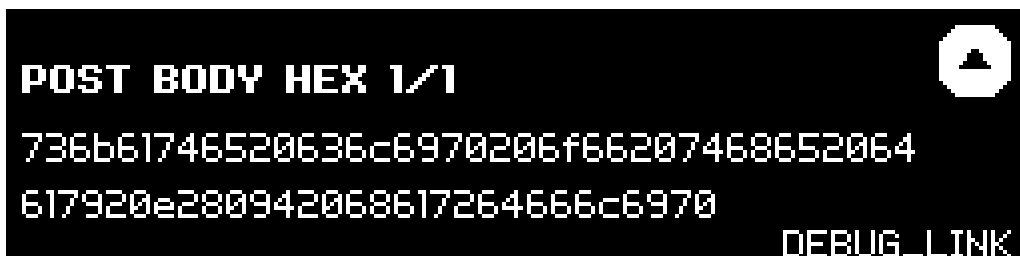
Parsed vote operation (test_msg_hive.py)
The device re-parses the Graphene bytes and displays voter, author, permalink and weight from the exact bytes being signed - a host serializer bug can only produce a rejection, never a silent wrong-sign.

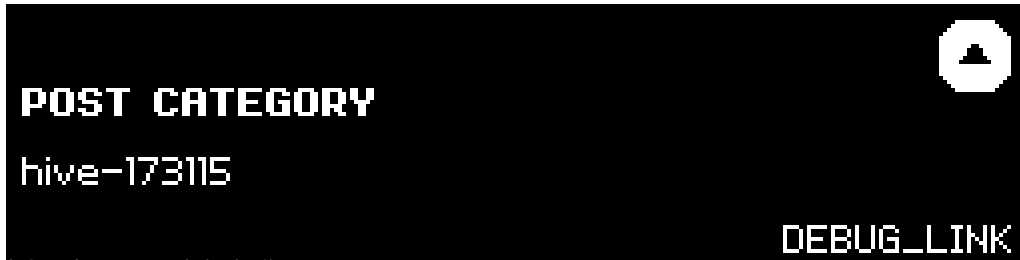
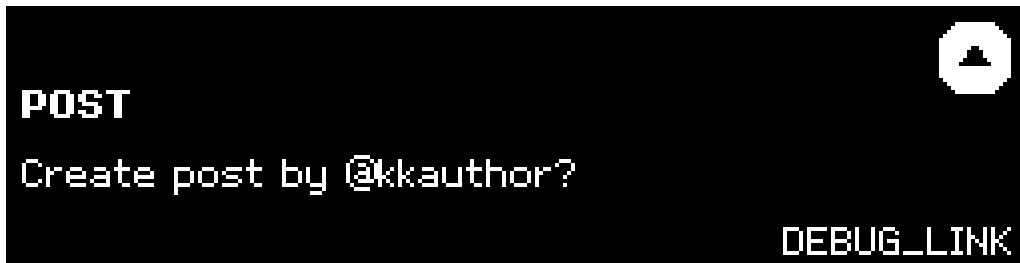


(3 OLED frames captured, showing 2)

✓G20 test_hive_sign_ops_comment

Parsed comment operation (test_msg_hive.py)
Comment title and body are user-controlled strings - both paged in full (72-char ASCII pages / complete hex for non-ASCII).



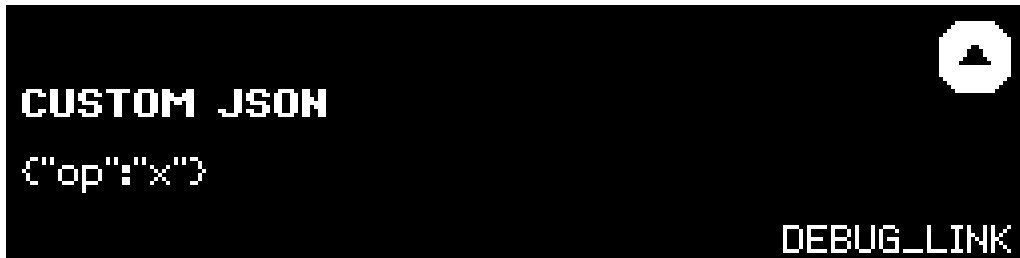
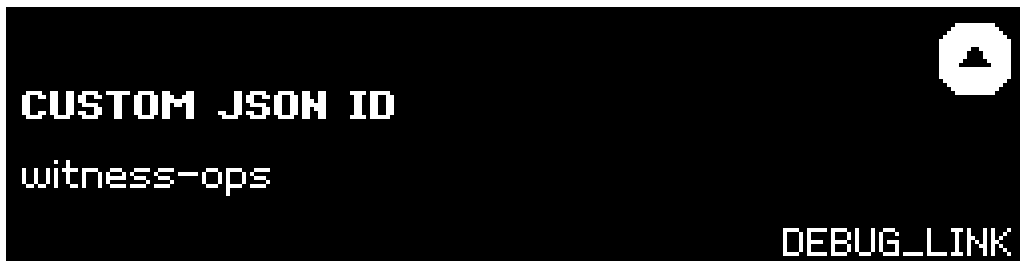


(7 OLED frames captured, showing 3)

✓G21 test_hive_sign_ops_custom_json_active

Parsed custom_json (active) (test_msg_hive.py)

custom_json id and payload paged in full under the active role.



(4 OLED frames captured, showing 3)

✓G22 test_hive_sign_ops_custom_json_posting

Parsed custom_json (posting) (test_msg_hive.py)

Same shape under the posting role (the common dApp path).

✓G23 test_hive_sign_ops_downvote_and_default_chain_id

Downvote + default chain id (test_msg_hive.py)

Negative weights display correctly and the default chain id binds the mainnet digest.

✓G24 test_hive_sign_ops_role_fences

Ops role fences (test_msg_hive.py)

vote/comment sign under posting; custom_json under its declared auth; memo' and owner' never sign operations.

✓G25 test_hive_sign_ops_rejects_excluded_and_unknown_ops

Unknown/excluded ops refused (no blind-sign) (test_msg_hive.py)

transfer-shaped and unrecognized operations inside SignOperations are refused - there is no blind-sign fallback for Graphene bytes the device cannot display.

✓G26 test_hive_sign_ops_rejects_malformed_structure

Malformed Graphene structure refused (test_msg_hive.py)
Truncated fields, wrong op counts and trailing bytes are all parse failures, not sign-what-you-can.

✓G27 test_hive_sign_ops_rejects_oversize

Oversize operations refused (test_msg_hive.py)
Payloads beyond the proto cap are refused before parsing.

✓G28 test_hive_sign_account_ops_reject_non_owner_roles

Account authority ops require owner (test_msg_hive.py)
account_create / account_update sign only under the owner role.

✓G29 test_hive_sign_ops_limit_order_create

Internal market: limit_order_create (test_msg_hive.py)
The op that motivated phase 3 - a HIVE->HBD market swap. Both sides of the order are shown with their symbols pinned (a swapped symbol hides a ~2000x value difference behind an identical-looking number), and order id / fill-or-kill / expiry get their own screen so they cannot be crowded off the first.

ORDER TERMS



Order #42

Fill or kill

Expires 1700003600

DEBUG_LINK

MARKET ORDER



@kktrader sells

1.5 HIVE

for >= 0.4 HBD

DEBUG_LINK

(3 OLED frames captured, showing 2)

✓G30 test_hive_sign_ops_limit_order_cancel

Internal market: limit_order_cancel (test_msg_hive.py)
Cancelling names the order id and the owner. No recipient row is forged - the op acts on the signer's own book entry.

CANCEL ORDER



Cancel order #42

for @kktrader?

DEBUG_LINK

SIGN TRANSACTION



Sign 1 Hive operation with the active key?

DEBUG_LINK

✓G31 test_hive_sign_ops_active_tier_value_ops

Active-tier value ops (test_msg_hive.py)
transfer_to_vesting, convert, transfer_to/from_savings, delegate_vesting_shares and withdraw_vesting all move or lock value, so all six sign only under active. Each renders its own amount + counterparty.

SAVINGS WITHDRAW

1.5 HIVE
from @kkuser
to @kkfriend

DEBUG_LINK



POWER UP

1 HIVE
from @kkuser
to @kkuser

DEBUG_LINK



CONVERT

Convert 2.5 HBD
to HIVE for @kkuser
(#7)

DEBUG_LINK



(13 OLED frames captured, showing 3)

✓G32 test_hive_sign_ops_posting_tier_ops

claim_reward_balance is posting tier (test_msg_hive.py)

Claiming is not spending, so it signs under posting. Three reward assets across two screens
(the OLED body fits three rows; a fourth would be signed but never shown).

CLAIM REWARDS

@kkuser claims
1.234 HIVE
5.678 HBD

DEBUG_LINK



CLAIM REWARDS

@kkuser claims
90.123456 VESTS

DEBUG_LINK



(3 OLED frames captured, showing 2)

✓G33 test_hive_sign_ops_zero_amount_semantics

Zero means something for two ops, nothing for the rest (test_msg_hive.py)

0 VESTS stops a power-down and removes a delegation - both legitimate, so zero is NOT rejected
there and the screen must say which action it is. Everywhere else a zero amount is a no-op and
refused.

REMOVE DELEGATION

@kkuser removes its
delegation to @kkfriend?

DEBUG_LINK





STOP POWER DOWN

Cancel power down
for @kkuser

DEBUG_LINK

(6 OLED frames captured, showing 2)

✓G34 test_hive_sign_ops_asset_symbol_and_precision_pinned

Asset symbol pinned to its protocol precision (test_msg_hive.py)

HIVE/HBD are 3-decimal, VESTS is 6. The parser refuses any other pairing: a wrong precision moves the decimal point on the confirmation screen relative to what the chain applies.

✓G35 test_hive_sign_ops_comment_options_binds_to_its_comment

comment_options binds to its own comment (test_msg_hive.py)

Payout redirection is only accepted immediately after a comment op with the same author and permalink. Standing alone it could attach beneficiaries to a post the user published earlier and is not reviewing on this screen.



SIGN TRANSACTION

Sign 2 Hive operations with the posting
key?

DEBUG_LINK



POST

Create post by @kkauthor?

DEBUG_LINK



POST CATEGORY

hive-100

DEBUG_LINK

(14 OLED frames captured, showing 3)

✓G36 test_hive_sign_ops_comment_options_beneficiary_rules

Beneficiary ordering, uniqueness and total enforced on-device (test_msg_hive.py)

Beneficiaries must be strictly ascending by account (which also makes them unique) and their weights must sum to no more than 10000 bp. Unsorted, duplicate and over-100% lists are each refused.

✓G37 test_hive_sign_ops_account_update2_rejects_authority_change

account_update2 cannot rotate keys (test_msg_hive.py)

Only the profile-metadata form is in the table. Any owner/active/posting/memo_key field present is a hard reject - the op-9/10 device-derived-keys invariant applied field-level.

✓G38 test_hive_sign_ops_truncated_bodies_rejected

Truncated op bodies refused (test_msg_hive.py)

A body cut short mid-field is a parse failure, not sign-what-you-can.

S. Solana [NEW]

NEW: Full Solana with Ed25519 (SLIP-10), base58 addresses, 37 instruction types across 7 programs. Key security fix: full 44-character address display replaces old 8-char truncation that was a spoofing vector.

User Flow

ADDRESS: m/44'/501'/0' Ed25519 -> full 44-char base58 on OLED

SIGN TX: Parse instructions -> per-instruction confirmation -> Ed25519 sign

SIGN MESSAGE: Arbitrary bytes -> hex display -> Ed25519 sign

Tests: 26/26 -- ALL PASSED

✓S1 test_solana_get_address

Derive Solana address (test_msg_solana_getaddress.py)

Full 44-character base58 address displayed on OLED.

OLED needed: Full 44-char address

✓S2 test_solana_different_accounts

Different account indices (test_msg_solana_getaddress.py)

Verifies different accounts produce different addresses.

✓S3 test_solana_deterministic

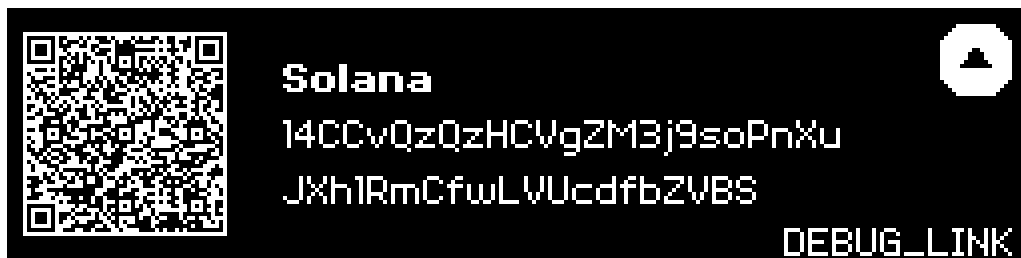
Deterministic derivation (test_msg_solana_getaddress.py)

Same path always produces same address.

✓S3b test_solana_show_address

Show address on OLED (test_msg_solana_getaddress.py)

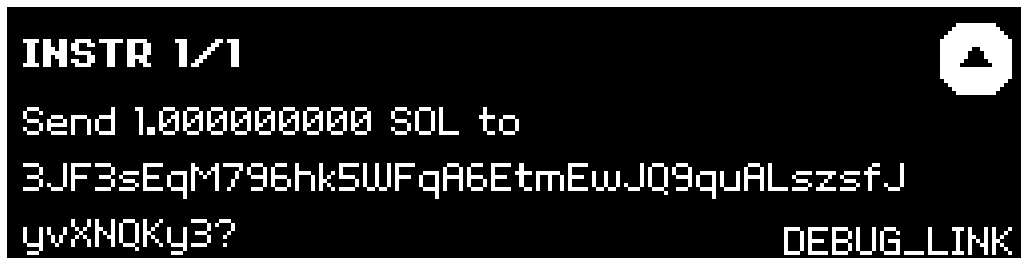
Full 44-char base58 address with QR code on OLED display.



✓S4 test_solana_sign_system_transfer

Sign SOL transfer (test_msg_solana_signtx.py)

System::Transfer with full address + amount display.



(2 OLED frames captured, showing 1)

✓S5 test_solana_sign_message

Sign Solana message (test_msg_solana_signtx.py)

Arbitrary message signing with Ed25519 key. Requires AdvancedMode policy (no domain separation).



(3 OLED frames captured, showing 1)

✓S6 test_solana_sign_empty_rejected

Empty tx rejected (test_msg_solana_signtx.py)

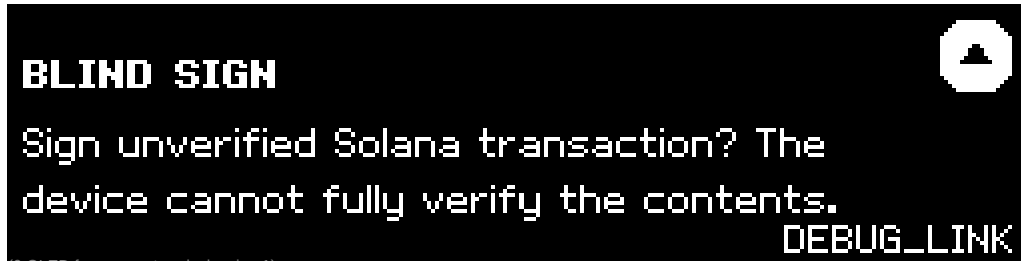
Zero-length transaction data is refused.

✓S7 test_solana_sign_deterministic

Deterministic signing (test_msg_solana_signtx.py)
Same tx always produces same signature.

✓S8 test_solana_sign_token_transfer

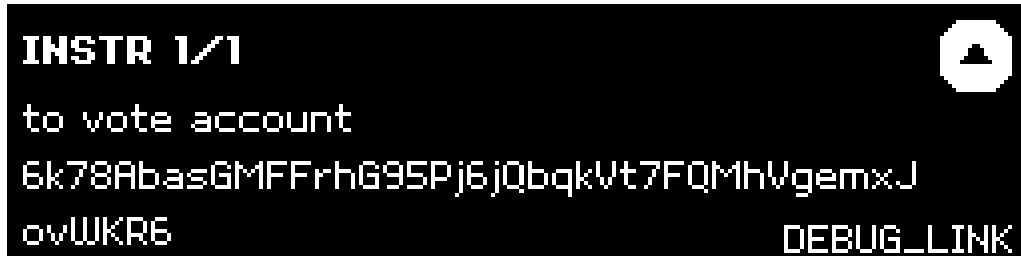
Unchecked SPL Transfer requires AdvancedMode (test_msg_solana_signtx.py)
Unchecked Transfer (op 3) carries NO signed mint - the device cannot prove which token is moving, so it is forced through the AdvancedMode blind-sign gate (matching Trezor and Ledger, which both reject it). Only TransferChecked clear-signs.



(6 OLED frames captured, showing 1)

✓S9 test_solana_sign_stake_delegate

Stake delegate (test_msg_solana_signtx.py)
Delegate SOL to a validator for staking rewards. OLED shows delegate confirmation.



(3 OLED frames captured, showing 2)

✓S10 test_solana_sign_memo

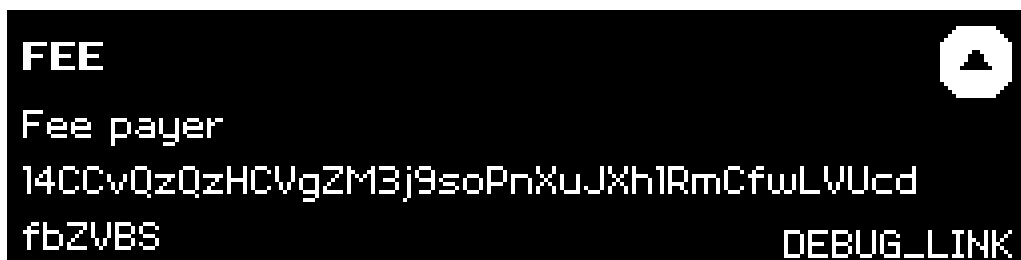
Memo instruction (test_msg_solana_signtx.py)
Attach memo text to transaction. OLED shows memo content.



(2 OLED frames captured, showing 1)

✓S11 test_solana_sign_compute_budget_unit_price

Compute budget unit price (test_msg_solana_signtx.py)
Set priority fee for transaction. OLED shows compute unit price.





INSTR 1/1

Set compute unit price to 50000?

DEBUG_LINK



FEE

Max priority fee
0.000070000 SOL

DEBUG_LINK

(4 OLED frames captured, showing 3)

✓S12 test_solana_sign_token_transfer_with_metadata

Host metadata does NOT bypass the unchecked-transfer gate (test_msg_solana_signtx.py)
An unchecked Transfer accompanied by host SolanaTokenInfo still requires AdvancedMode: the mint is not part of the signed instruction, so the metadata is unauthenticated and must not make the tx look clear-signable.



BLIND SIGN

Sign unverified Solana transaction? The
device cannot fully verify the contents.

DEBUG_LINK

(6 OLED frames captured, showing 1)

✓S13 test_solana_sign_token_transfer_checked

TransferChecked clear-signs with the mint on its own screen (test_msg_solana_signtx.py)
TransferChecked (op 12) binds the mint in the signed instruction bytes. The device shows "Token mint <full base58>" on a DEDICATED screen before the amount - the authenticated token identity cannot be pushed off-view by a host-controlled symbol - and decimals come from the signed instruction, never from the host. AdvancedMode stays OFF.



INSTR 1/1

Known token USDC
EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZ
wyTDtlv

DEBUG_LINK



INSTR 1/1

Send 1.5 USDC to
4Ss5JmKXAD9Z7cktFEdrqeMuT6jGMFlpVozTyP
HZ6zT4?

DEBUG_LINK

(4 OLED frames captured, showing 2)

✓S14 test_solana_sign_token_transfer_checked_attested_symbol

Signed token definition: symbol attested by a loaded signer (test_msg_solana_signtx.py)
The token_info carries a secp256k1 attestation over (mint, decimals, symbol) by a signer loaded via LoadClearsignSigner - the same chain-agnostic trust anchor as EVM clear-sign metadata (KeepKey's open equivalent of Trezor's CoSi-signed token definitions). The device verifies it, requires the attested decimals to equal the signed instruction's, and adds a "Token "USDC" signed by <alias> <fingerprint>" screen. An invalid attestation rejects the symbol outright (never falls back to the claim).

LOAD CLEARSIGNER



Trust 'CI Test' (0C2CB8B9) for this session
to describe transactions? NOT verified by
KeepKey.

[DEBUG_LINK](#)

INSTR 1/1



Known token USDC
EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZ
wyTDt1v

[DEBUG_LINK](#)

INSTR 1/1



Send 1.5 USDC to
4Ss5JmKXAD9Z7cktFEdrqeMuT6jGMFlpVozTyP
HZ6zT4?

[DEBUG_LINK](#)

(6 OLED frames captured, showing 3)

✓S15 test_solana_sign_token_approve

Unchecked SPL Approve requires AdvancedMode (test_msg_solana_signtx.py)
Approve (op 4) hides the delegated token's mint - same gate as unchecked Transfer.

BLIND SIGN



Sign unverified Solana transaction? The
device cannot fully verify the contents.

[DEBUG_LINK](#)

(6 OLED frames captured, showing 1)

✓S16 test_solana_sign_create_account_requires_advanced_mode

CreateAccount requires AdvancedMode (test_msg_solana_signtx.py)
CreateAccount assigns the new account's owner program and space, which the screen does not
fully disclose - gated rather than partially clear-signed.

BLIND SIGN



Sign unverified Solana transaction? The
device cannot fully verify the contents.

[DEBUG_LINK](#)

(6 OLED frames captured, showing 1)

✓S17 test_solana_sign_set_authority_requires_advanced_mode

SetAuthority requires AdvancedMode (test_msg_solana_signtx.py)
SetAuthority hands over control of a mint/account (including the undistinguishable "clear
authority" case) - an account-takeover vector, gated.



BLIND SIGN

Sign unverified Solana transaction? The device cannot fully verify the contents.

[DEBUG_LINK](#)

(6 OLED frames captured, showing 1)

✓S18 test_solana_sign_stake_authorize_clearsigns

StakeAuthorize clear-signs role + new authority (test_msg_solana_signtx.py)

Shows the stake account, the role being reassigned (staker/withdrawer) and the full new authority address.



INSTR 1/1 1/2

Authorize staker
to

AByCTxLPRZPoyK22KdMxa3xkCbcdNbeNUgHwE5ZLINK



INSTR 1/1

Stake account

14CCvQzQzHCVgZM3j9soPnXuJXh1RmCfwLVUcd
fbZVBS

[DEBUG_LINK](#)



INSTR 1/1 2/2

h6UcJs9?

[DEBUG_LINK](#)

(4 OLED frames captured, showing 3)

✓S19 test_solana_sign_stake_withdraw

Stake withdraw shows the destination (test_msg_solana_signtx.py)

The withdrawal destination account is displayed in full - a host cannot silently redirect withdrawn SOL.



INSTR 1/1 1/2

Withdraw 2.000000000 SOL
to

6k78AbasGMFFrhG95Pj6jQbqkVt7FQMHVg5B06LINK

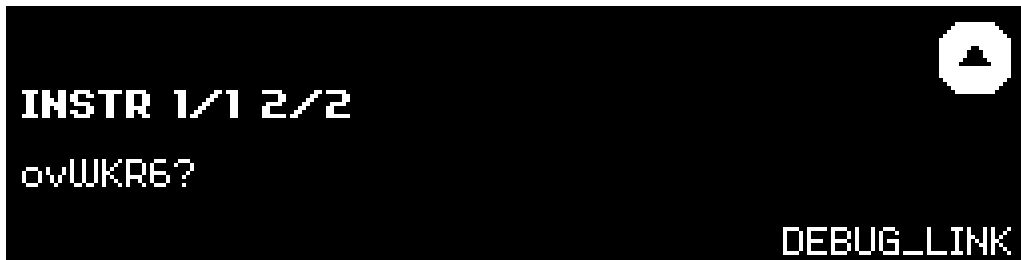


INSTR 1/1

Withdraw from stake

14CCvQzQzHCVgZM3j9soPnXuJXh1RmCfwLVUcd
fbZVBS

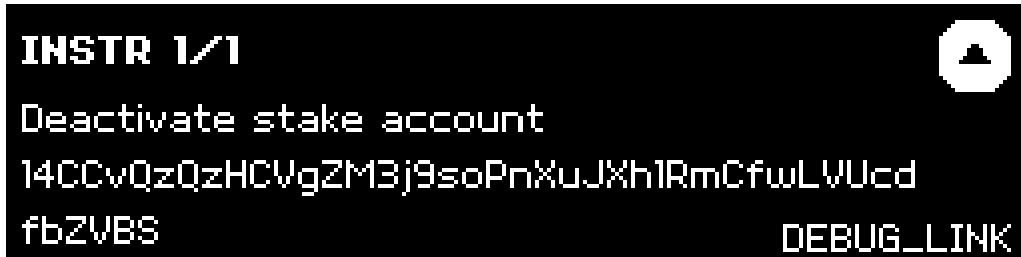
[DEBUG_LINK](#)



(4 OLED frames captured, showing 3)

✓S20 test_solana_sign_stake_deactivate

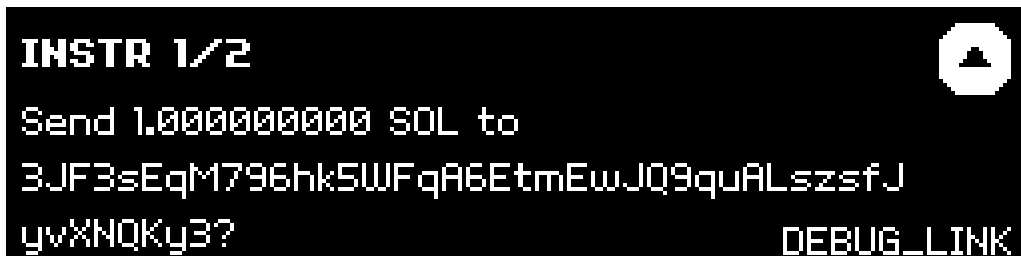
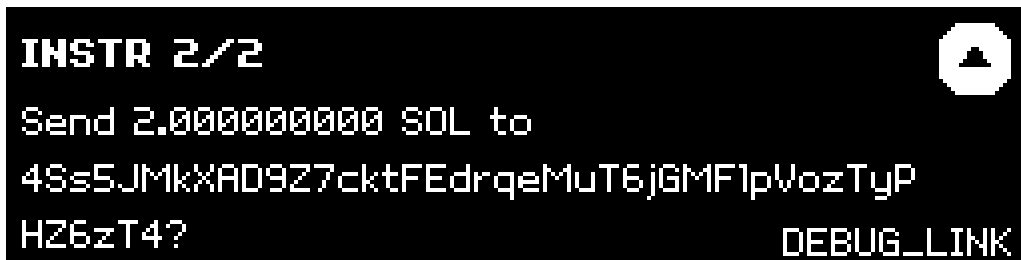
Stake deactivate shows the stake account (test_msg_solana_signtx.py)
The acted-on stake account is named on-screen.



(2 OLED frames captured, showing 1)

✓S21 test_solana_sign_multi_instruction_2x_transfer

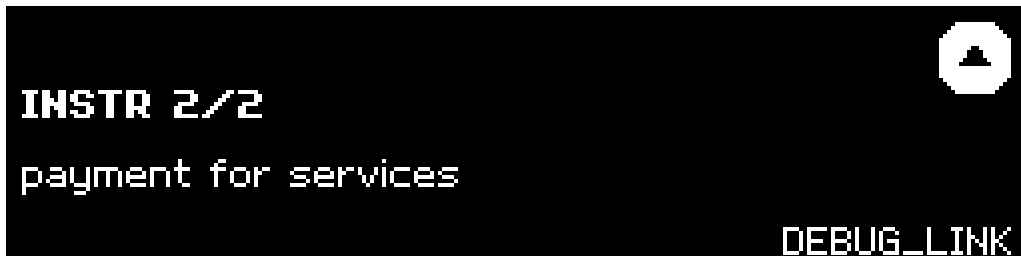
Multi-instruction: each instruction confirmed (test_msg_solana_signtx.py)
Two transfers in one tx produce INSTR 1/2 and INSTR 2/2 screens - nothing rides along
unconfirmed.



(3 OLED frames captured, showing 2)

✓S22 test_solana_sign_multi_instruction_transfer_and_memo

Transfer + memo both shown (test_msg_solana_signtx.py)
A transfer with an attached memo instruction confirms both.



INSTR 1/2

Send 1.000000000 SOL to
3JF3sEqM796hk5WFqA6EtmEwJQ9quALszsfJ
yvXNQKy3?

DEBUG_LINK

(3 OLED frames captured, showing 2)

✓S23 test_solana_sign_versioned_v0_static_verified

Versioned (v0) tx with static keys clear-signs (test_msg_solana_signtx.py)

A v0-format tx whose accounts are all static parses and clear-signs like legacy.

INSTR 1/1

Send 1.000000000 SOL to
3JF3sEqM796hk5WFqA6EtmEwJQ9quALszsfJ
yvXNQKy3?

DEBUG_LINK

(3 OLED frames captured, showing 1)

✓S24 test_solana_sign_versioned_v0_opaque

v0 with address-table lookups requires AdvancedMode (test_msg_solana_signtx.py)

Lookup-table accounts cannot be resolved on-device, so the tx routes to the blind-sign gate.

✓S25 test_solana_sign_x402_zero_lut_usdc_payment

x402 zero-LUT v0 USDC payment is hardware verified (test_msg_solana_signtx.py)

The sponsor pays fees while the KeepKey key authorizes TransferChecked. The device renders 0.002 USDC from firmware-owned mint metadata, derives ATA(payTo, mint) offline, and displays the merchant owner only after it matches the signed destination token account. The required x402 uniqueness memo is also displayed; AdvancedMode stays OFF.

INSTR 3/4

Verified recipient owner
GmaDrppBC7P5ARKV8g3djiwP89vz1jLK23V2GBj
uAEGB

DEBUG_LINK

INSTR 1/4

Set compute unit limit to 120000?

DEBUG_LINK

INSTR 2/4

Set compute unit price to 1000?

DEBUG_LINK

(10 OLED frames captured, showing 3)

T. TRON [NEW]

NEW: TRON with secp256k1 signing, base58 addresses. Blind-sign via raw_data. Structured reconstruct-then-sign and TRC-20 clear-signing deferred to a future release.

User Flow

ADDRESS: m/44'/195'/0'/0/0 -> full 34-char base58 TRON address

BLIND-SIGN: Raw protobuf data -> hash + sign

Tests: 6/6 -- ALL PASSED

✓T1 test_tron_get_address

Derive TRON address (test_msg_tron_getaddress.py)

Full 34-character base58 address.

OLED needed: Full 34-char address

✓T2 test_tron_different_accounts

Different accounts (test_msg_tron_getaddress.py)

Different indices produce different addresses.

✓T3 test_tron_deterministic

Deterministic derivation (test_msg_tron_getaddress.py)

Same path always produces same address.

✓T3b test_tron_show_address

Show address on OLED (test_msg_tron_getaddress.py)

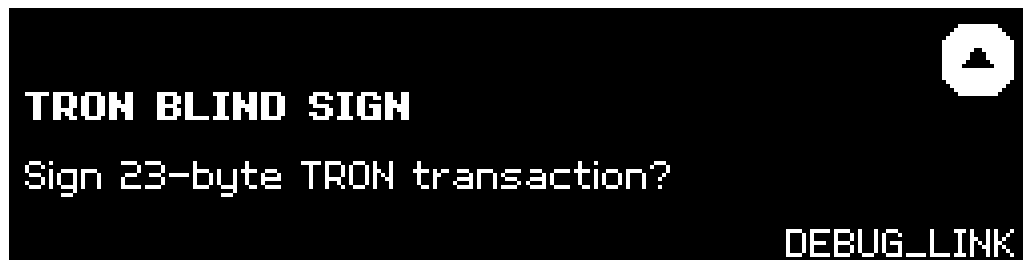
Full 34-char Base58Check TRON address with QR code.



✓T4 test_tron_sign_transfer_legacy_raw_data

Sign TRX blind (raw_data) (test_msg_tron_signtx.py)

Raw protobuf data triggers blind sign path. Shows amount + address if provided.



(3 OLED frames captured, showing 1)

✓T5 test_tron_sign_missing_fields_rejected

Missing fields rejected (test_msg_tron_signtx.py)

Incomplete transaction data is refused.

N. TON [NEW]

NEW: TON v4r2 wallet contracts. Ed25519 signing with structured field display. Blind-sign for raw transactions. Memo/comment support. Full clear-sign with cell tree reconstruction deferred to a future release.

User Flow

ADDRESS: m/44'/607'/0' -> full 48-char base64url TON address

STRUCTURED: Amount + address + memo shown as display context -> sign

BLIND-SIGN: Raw tx without structured fields -> "BLIND SIGNATURE" warning

Tests: 8/8 -- ALL PASSED

✓N1 test_ton_get_address

Derive TON address (test_msg_ton_getaddress.py)

Full 48-character base64url address.

OLED needed: Full 48-char address

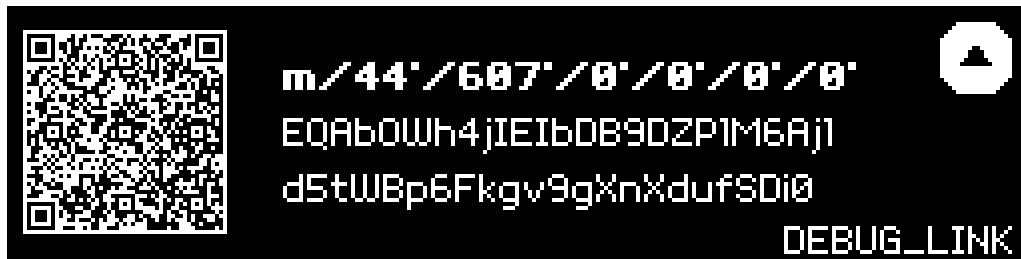
✓N2 test_ton_different_accounts

Different accounts (test_msg_ton_getaddress.py)

Different indices produce different addresses.

✓N2b test_ton_show_address

Show address on OLED (test_msg_ton_getaddress.py)
Full 48-char base64url TON address with QR code.

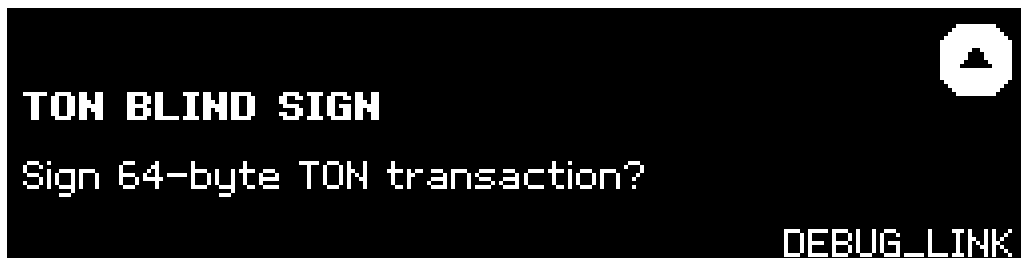


✓N3 test_ton_address_format

Address format validation (test_msg_ton_getaddress.py)
Bounceable/non-bounceable format check.

✓N4 test_ton_sign_structured

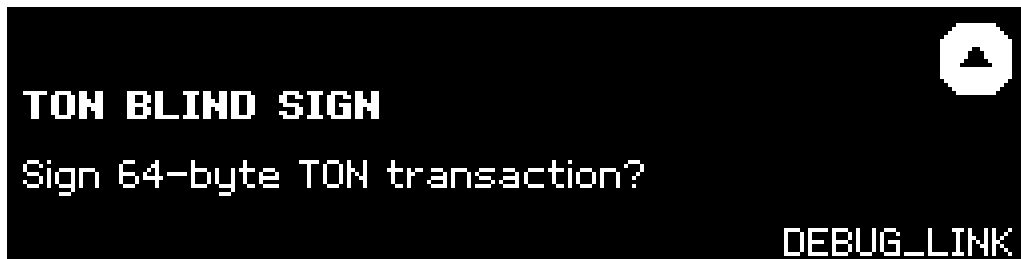
Sign TON transfer (test_msg_ton_signtx.py)
Structured fields shown as display context. Blind-sign with amount + address.



(2 OLED frames captured, showing 1)

✓N5 test_ton_sign_with_memo

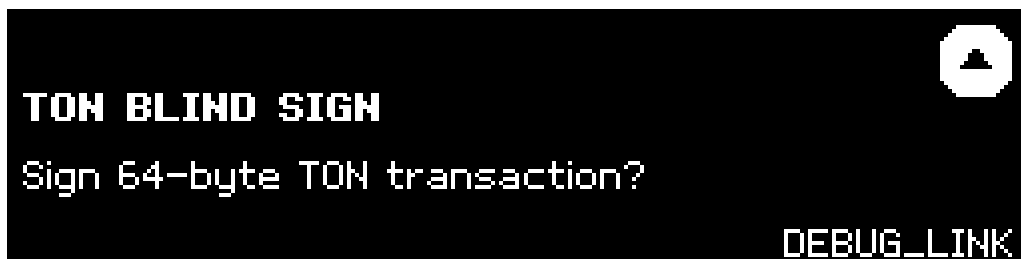
Sign TON with memo (test_msg_ton_signtx.py)
Memo/comment displayed before signing.



(2 OLED frames captured, showing 1)

✓N6 test_ton_sign_legacy_raw_tx

Sign TON blind (test_msg_ton_signtx.py)
Raw tx without structured fields triggers blind sign.



(2 OLED frames captured, showing 1)

✓N7 test_ton_sign_missing_fields_rejected

Missing fields rejected (test_msg_ton_signtx.py)
Incomplete data refused.

Y. Zcash Transparent

Transparent t-address Zcash (send/receive) over the generic Bitcoin UTXO signing path with Overwinter/Sapling-v4 branch handling. This is the Zcash functionality that ships ENABLED on the regular 7.15.0 build -- t1.../t3... addresses sign like Bitcoin (SECP256K1) with a FeeOverThreshold guard. No shielded/Orchard engine is involved; contrast with section Z (shielded), which also ships in the regular product and is stripped from bitcoin-only.

User Flow

INPUT: TxInputType over the Zcash coin (t-address, SECP256K1)
METADATA: version_group_id + branch_id for the target upgrade
CONFIRM: amount + destination on the OLED, then sign each input
FEE GUARD: an implausibly high fee triggers a confirmation prompt

Tests: 4/4 -- ALL PASSED

✓1 test_transparent_one_one

Transparent 1-in 1-out (test_msg_signtx_zcash.py)

Sign a standard transparent Zcash spend; the device shows the amount and destination t-address before producing a signature over the overwinter sighash.

TRANSACTION

Do you want to send 9.99884999 TAZ from your wallet? This includes a transaction fee of 0.0000194 TAZ.

DEBUG_LINK

Send 9.99883059 TAZ to tmJlxYxP8XNTtCoDgvdmQPSrxh5qZJgy65Z

DEBUG_LINK

✓2 test_transparent_one_one_fee_too_high

High-fee guard (test_msg_signtx_zcash.py)

An implausibly high fee triggers the FeeOverThreshold confirmation before signing.

CONFIRM FEE

Really spend 9.0000194 TAZ on fees? Except in times of high network congestion, fees should be less than 10000 sat/byte.

DEBUG_LINK

Send 0.9999806 TAZ to tmJlxYxP8XNTtCoDgvdmQPSrxh5qZJgy65Z

DEBUG_LINK

TRANSACTION

Do you want to send 10.0 TAZ from your wallet? This includes a transaction fee of 9.0000194 TAZ.

DEBUG_LINK

✓3 test_shieldedIn_one_one_fee_1

Transparent spend (fee scenario 1) (test_msg_signtx_zcash.py)

Despite the legacy method name, this signs a transparent input/output over the same overwinter path (no Orchard).

✓V4 test_shieldedIn_one_one_fee_2

Transparent spend (fee scenario 2) (test_msg_signtx_zcash.py)
Second transparent fee scenario over the overwinter path.

Z. Zcash Shielded (Orchard) [NEW]

Shielded Orchard (PCZT streaming, Full Viewing Key export, unified-address display with an on-device ZIP-32 Sec 6.1 seed-fingerprint attestation) ships in the regular 7.15.0 product. KK_ZCASH_PRIVACY is enabled for the regular build and disabled only for bitcoin-only. This report covers device FVK/address behavior and the Python PCZT streaming contract. Mainnet proof construction and the physical shield, deshield, and Orchard-to-Orchard matrix are recorded separately in the RC18 release evidence.

User Flow

FVK: Derive ak, nk, rivk components via ZIP-32 Orchard path
ADDRESS: Device derives its own unified address + shows it; optional seed-fingerprint pin
PCZT: Stream header -> actions one at a time -> confirm each -> return signatures
HYBRID: Transparent inputs + Orchard outputs in same tx

Tests: 25/25 -- ALL PASSED

✓Z1 test_fvk_reference_vectors

FVK reference vectors (test_msg_zcash_orchard.py)
FVK output matches known test vectors.
OLED needed: FVK export

✓Z2 test_fvk_field_ranges

FVK field ranges (test_msg_zcash_orchard.py)
ak, nk, rivk are within valid Pallas curve ranges.

✓Z3 test_fvk_consistency_across_calls

FVK deterministic (test_msg_zcash_orchard.py)
Same account always produces same FVK.

✓Z4 test_fvk_different_accounts

FVK different accounts (test_msg_zcash_orchard.py)
Different accounts produce different FVKs.

✓Z5 test_fvk_abandon_mnemonic

FVK abandon-mnemonic vector (test_msg_zcash_orchard.py)
FVK derivation matches the Orchard reference vector for the standard abandon mnemonic.

✓Z6 test_zcash_display_address_basic

Display unified address (test_msg_zcash_display_address.py)
Device derives its OWN Orchard unified address (u1...) from the ZIP-32 path, shows it on the OLED for confirmation, and returns it with the device seed fingerprint. The host does not supply the address - this defends against a compromised host showing a fake UA.

Zcash #0 Orchard
uluzslncvvrw4r2y2kgjz7fm477xcnzge9z45sc
m4e6l6c63ren0ru29teedxw5vxu7c8xchp3ec2
pu3wkgldc5zphwtm4w3fchcwrl26c
DEBUG_LINK

**Zcash #0 Orchard**
DEBUG_LINK

✓Z7 test_zcash_display_address_bad_path_rejected

Reject malformed address path (test_msg_zcash_display_address.py)
A path that is neither m/32'/133'/account' nor an explicit account is rejected with a SyntaxError, so no wrong-account address is ever derived silently.

✓Z8 test_get_orchard_fvk_returns_seed_fingerprint

FVK carries seed fingerprint (test_msg_zcash_seed_fingerprint.py)
ZcashGetOrchardFVK returns a 32-byte ZIP-32 §6.1 seed fingerprint alongside the FVK.

✓Z9 test_fingerprint_stable_across_accounts

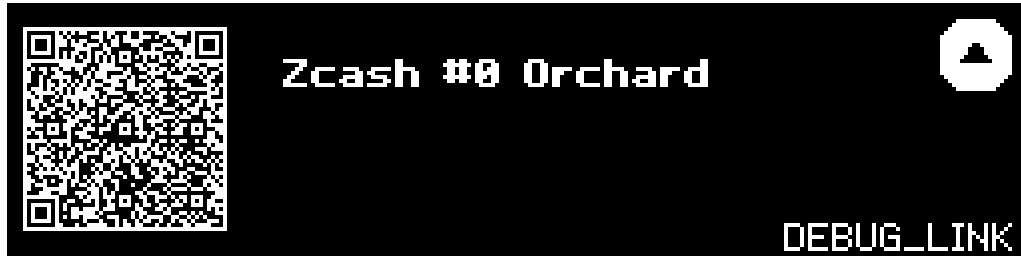
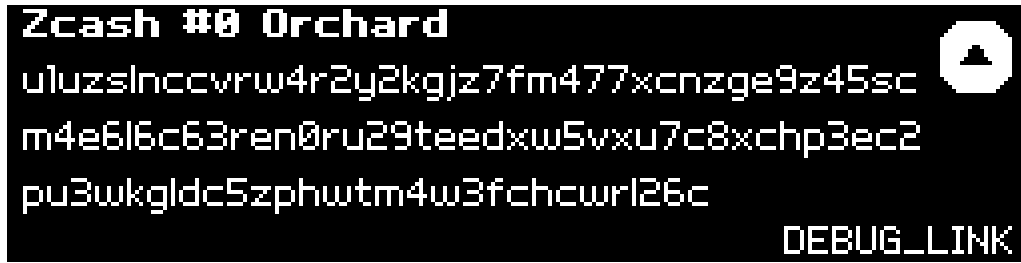
Fingerprint bound to seed not account (test_msg_zcash_seed_fingerprint.py)

The seed fingerprint is identical across account indices - it identifies the device seed.

✓Z10 test_display_address_helper_accepts_matching_fingerprint

Address display accepts matching fingerprint (test_msg_zcash_seed_fingerprint.py)

When the host supplies expected_seed_fingerprint and it matches, the device derives and displays the address and echoes the fingerprint.



✓Z11 test_display_address_helper_rejects_wrong_fingerprint

Address display rejects wrong fingerprint (test_msg_zcash_seed_fingerprint.py)

A mismatched expected_seed_fingerprint is rejected before any derivation - the host cannot get an attestation from the wrong device.

✓Z12 test_display_address_helper_backward_compat

Address display without fingerprint (test_msg_zcash_seed_fingerprint.py)

Omitting expected_seed_fingerprint still works; the device populates the fingerprint on the response regardless.

✓Z13 test_device_fingerprint_matches_python_helper

Fingerprint matches host computation (test_msg_zcash_seed_fingerprint.py)

The device-derived fingerprint equals calculate_seed_fingerprint(seed) - firmware C and the python helper agree byte-for-byte for the all-all seed.

✓Z14 test_sign_pczt_helper_rejects_wrong_fingerprint

PCZT signing rejects wrong fingerprint (test_msg_zcash_seed_fingerprint.py)

A wrong expected_seed_fingerprint on a PCZT signing request is rejected before any signing crypto runs.

✓Z15 test_all_dummy_shield_streams_outputs_inputs_and_no_orchard_sigs

Shield streams dummy actions without device signatures (test_msg_zcash_sign_pczt.py)

The client streams transparent inputs/outputs and both dummy Orchard actions, preserves their finalized PCZT signatures, and expects no compact device Orchard signatures.

✓Z16 test_mixed_deshield_returns_only_real_spend_signature

Deshield returns only real-spend signatures (test_msg_zcash_sign_pczt.py)

A mixed real/dummy Orchard action set returns one compact signature for the real spend.

✓Z17 test_private_send_preserves_compact_real_spend_order

Private send preserves real-spend signature order (test_msg_zcash_sign_pczt.py)

Compact device signatures remain ordered by the real-spend actions when dummy actions are interleaved. OFFLINE CONTRACT TEST -- like every test in test_msg_zcash_sign_pczt, it drives a ScriptedTransport with canned responses and never reaches a device. It proves the client builds and orders the messages correctly; it proves nothing about firmware behaviour, and it can never produce an OLED frame. ZcashSignPCZT is not sent to a device anywhere in this suite, so the on-device shielded signing path -- including the per-output confirm that is the designed verification gate for Orchard output values -- has no automated coverage at all. Shielded signing must be walked on real hardware.

✓Z18 test_missing_is_spend_is_rejected_before_device_call

Missing spend classification rejected (test_msg_zcash_sign_pczt.py)

Every action must explicitly declare is_spend before any device call is made.

✓Z19 test_host_transparent_sighash_is_rejected_before_device_call

Host transparent sighash rejected (test_msg_zcash_sign_pczt.py)

The client refuses a host-provided transparent sighash instead of forwarding it as trusted device input.

✓Z20 test_signature_count_must_match_real_spends

Signature count bound to real spends (test_msg_zcash_sign_pczt.py)

The returned compact signature count must equal the number of real-spend actions.

✓Z21 test_duplicate_action_request_is_rejected

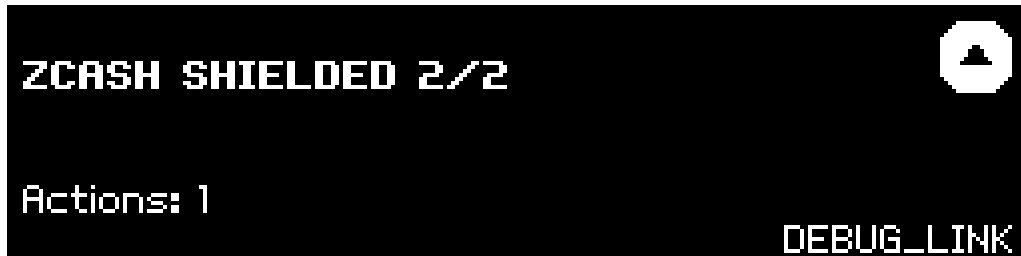
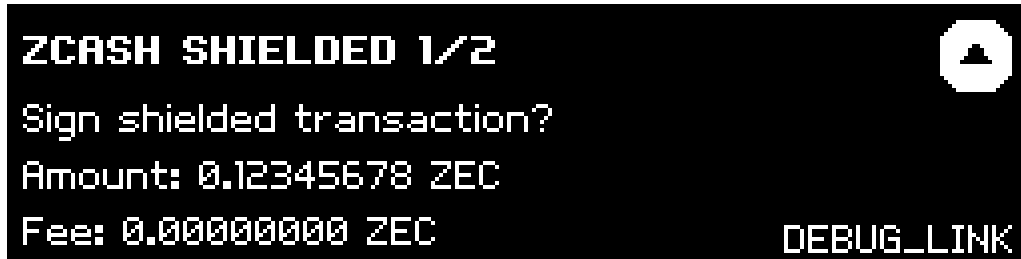
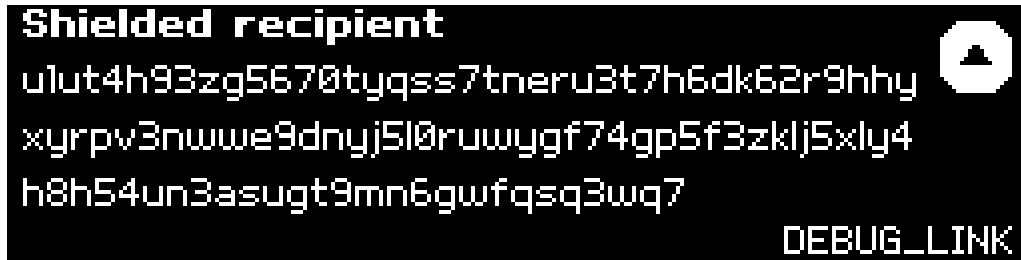
Duplicate action requests rejected (test_msg_zcash_sign_pczt.py)

A repeated device request for the same action index aborts the streaming session.

✓Z22 test_shielded_output_review_is_two_screens

Shielded output review: amount and full address (ON DEVICE) (test_msg_zcash_sign_pczt_device.py)

The first test in this suite that sends ZcashSignPCZT to an actual device -- Z15-Z21 above are offline contract tests against a scripted transport. Signs a shielded-only transaction built from the firmware's own known-answer note vector, so the device accepts its recomputed commitment, and asserts the output review is two screens. It has to be: a unified address is 106 characters, three full body rows, and the body is three rows total, so a single confirm holding the question, the address and the amount renders 76 characters of address and silently drops the rest along with the amount. That screen is the verification gate for Orchard output values -- total_amount on the summary is a host-supplied prompt -- so the amount vanishing there is the whole trust story. Verified as a regression test: against the shipped 7.15.0 RC emulator it fails with "expected 2 ConfirmOutput screens, got 1".



(5 OLED frames captured; showing 3)

✓Z23 test_note_commitment_binds_the_recipient

Tampered recipient breaks the note commitment (ON DEVICE) (test_msg_zcash_sign_pczt_device.py)

Flipping one bit of the recipient makes the device-recomputed cmx disagree with the supplied commitment, and signing is refused. This is what stops a host displaying one recipient while committing to another.

✓Z24 test_pool_selection_is_honoured

Orchard commitment rejected under the Ironwood pool (ON DEVICE) (test_msg_zcash_sign_pczt_device.py)

The same note commits to a different value in each pool, so offering the Orchard commitment while declaring Ironwood must be rejected. Passes trivially if the device ignores shielded_pool, which is why it is paired with Z25.

✓Z25 test_ironwood_note_is_accepted

Ironwood commitment for the same note is accepted (ON DEVICE) (test_msg_zcash_sign_pczt_device.py)

The positive half of Z24: identical inputs, Ironwood commitment, accepted. Together they prove the pool branch is selected by shielded_pool rather than one path serving both.

D. BIP-85 Child Derivation [NEW]

NEW: Derives child BIP-39 mnemonic from master seed via HMAC-SHA512 (BIP-85). Display-only: derived words appear on OLED, never transmitted over USB. Seed accessed in CONFIDENTIAL buffer, memzero'd after use.

User Flow

DERIVE: word_count + language + index -> HMAC-SHA512 -> child entropy -> BIP-39 words

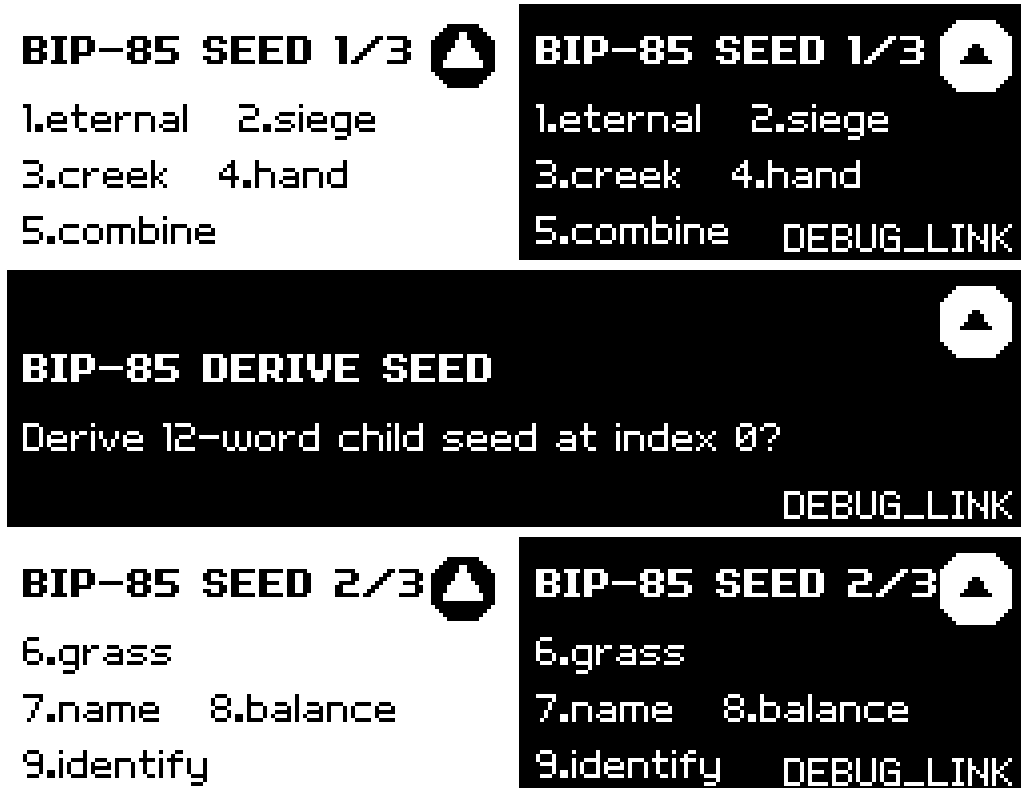
DISPLAY: Words shown on OLED only -> user writes down -> never sent to host

Tests: 6/6 -- ALL PASSED

✓D1 test_bip85_12word_flow

Derive 12-word child (test_msg_bip85.py)

Derives 128 bits of child entropy -> 12-word BIP-39 mnemonic displayed on OLED.



(4 OLED frames captured, showing 3)

✓D2 test_bip85_24word_flow

Derive 24-word child (test_msg_bip85.py)

256 bits -> 24 words.

✓D3 test_bip85_18word_flow

Derive 18-word child (test_msg_bip85.py)

192 bits -> 18 words.

✓D4 test_bip85_different_indices_different_flows

Different indices (test_msg_bip85.py)

Index 0 and index 1 must produce completely different mnemonics.

✓D5 test_bip85_deterministic_flow

Deterministic (test_msg_bip85.py)

Same seed + same index always produces same child mnemonic.

✓D6 test_bip85_invalid_word_count

Invalid count rejected (test_msg_bip85.py)

Word counts other than 12/18/24 are refused.

Appendix: Device Specifications

The KeepKey is an open-source hardware wallet built on an ARM Cortex-M3 (STM32F205, 120MHz) with a 256x64 monochrome OLED, single confirmation button, and micro-USB interface. The bootloader (v2.x) is flashed at manufacture and never updated - it is the immutable root of trust. On every boot, the bootloader verifies the firmware signature using redundant F3 checks before transferring control.

BOOT SEQUENCE:

1. USB connect -> bootloader executes (always first)
2. F3 signature check (redundant dual-path verify)
3. Valid -> KeepKey logo -> firmware runs

4. Invalid/missing -> "UPDATE FIRMWARE" screen
5. Firmware upload -> verify -> flash -> reboot -> re-verify

HARDWARE:

- MCU: STM32F205RET6, 120MHz, 128KB bootloader + 896KB firmware
- Display: 256x64 OLED (SSD1306), monochrome, used for ALL confirmations
- Input: single capacitive button (confirm/reject)
- USB: micro-B, HID + WebUSB transports, HID fallback
- Storage: BIP-39 seed encrypted in isolated flash region
- Curves: secp256k1, ed25519, NIST P-256; regular firmware also includes Pallas/Orchard

SECURITY MODEL:

- All private key operations happen on-device, keys never leave
- Every transaction output displayed on OLED for user verification
- PIN grid randomized on each prompt (position-based, not digit-based)
- BIP-39 passphrase creates hidden wallets (plausible deniability)

FIRMWARE VARIANTS (7.15, PR #282):

- Full multi-chain (default): all coin families including Zcash Orchard privacy; `firmware_variant` = model name.
- Bitcoin-only (`KK_BITCOIN_ONLY`): only Bitcoin + Testnet; all altcoin and shielded-Zcash handlers stripped; `firmware_variant` = `KeepKeyBTC` (EmulatorBTC on the emulator). Clients gate multi-chain-only tests on this string.
- There is no separate Zcash artifact: `KK_ZCASH_PRIVACY` is ON for the regular product and OFF only for `KK_BITCOIN_ONLY`.

SEED LOCK (7.15, PR #282):

- A seed created under bitcoin-only firmware is stamped in a reserved storage-version band. Multi-chain firmware refuses to load it and requires an explicit wipe (wipe-to-exit); the seed is never exposed to stripped-out code. Old multi-chain firmware treats the band as unknown and resets.